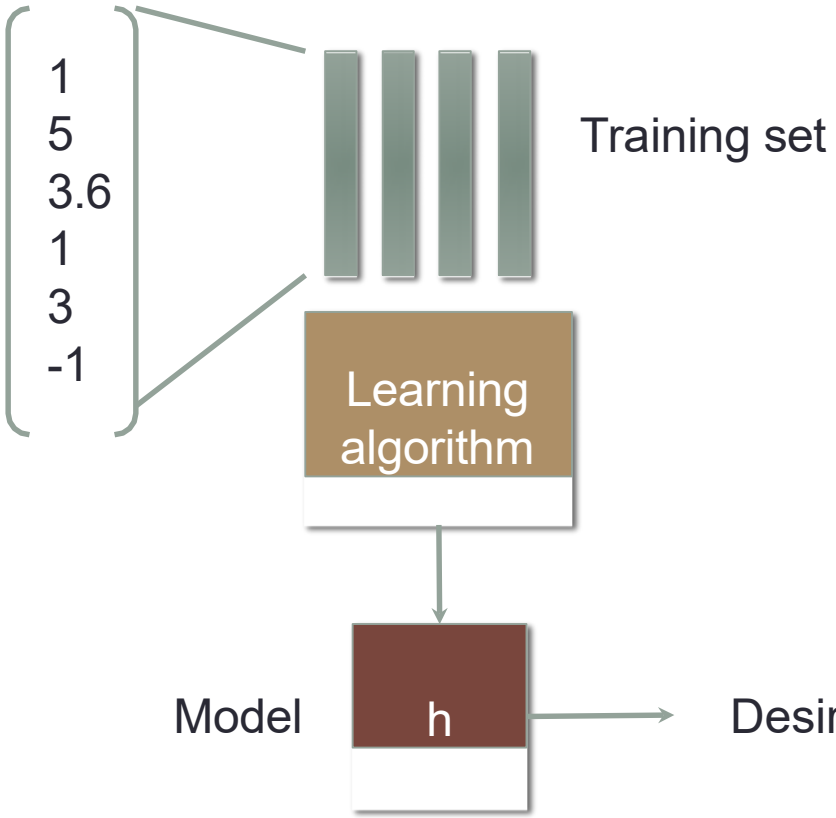
REGRESSION

with some K-mean clustering

How do we learn from data?



Training phase

Types of machine learning

1. Supervised learning

   Learn a model F from pairs of (**x**,y)

2. Unsupervised learning

   Discover the hidden structure in unlabeled data **x** (no y)

3. Reinforcement learning

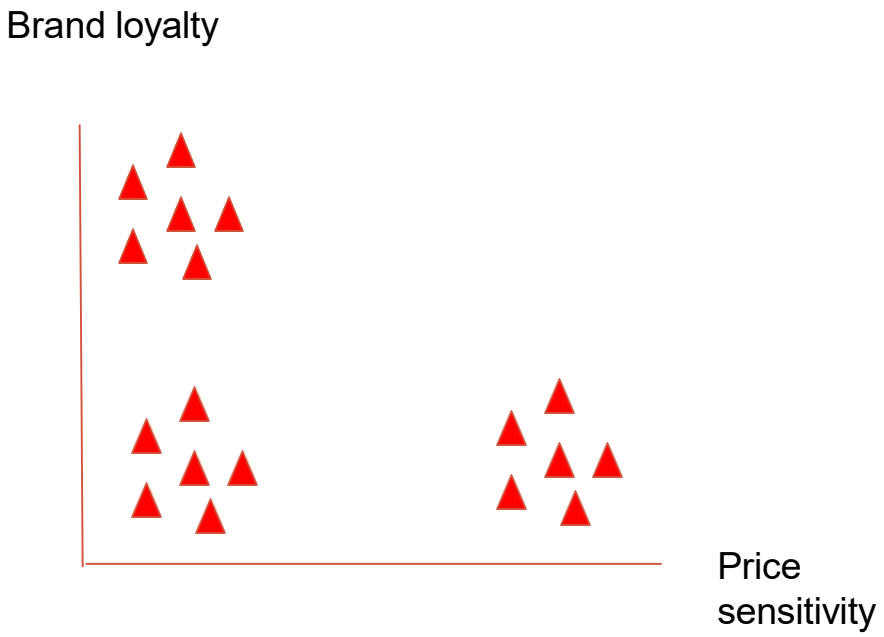
   Train an agent to take appropriate actions in an environment by maximizing rewards

Our first model - Unsupervised learning

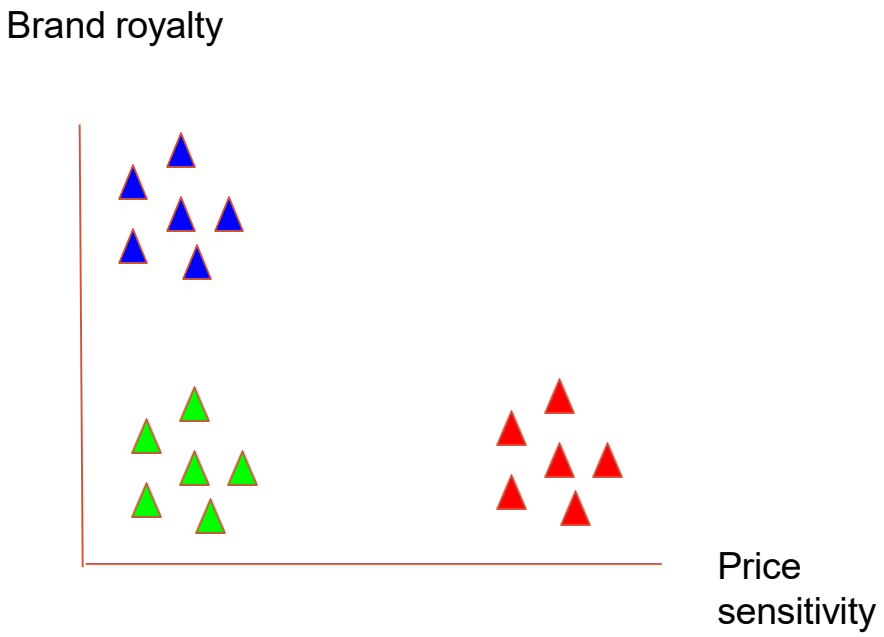
Discover the hidden structure in unlabeled data X (no y)

- Customer/product segmentation
- Data analysis for ...
- Identify number of speakers in a meeting recording
- Helps supervised learning in some task

Example - Customer analysis

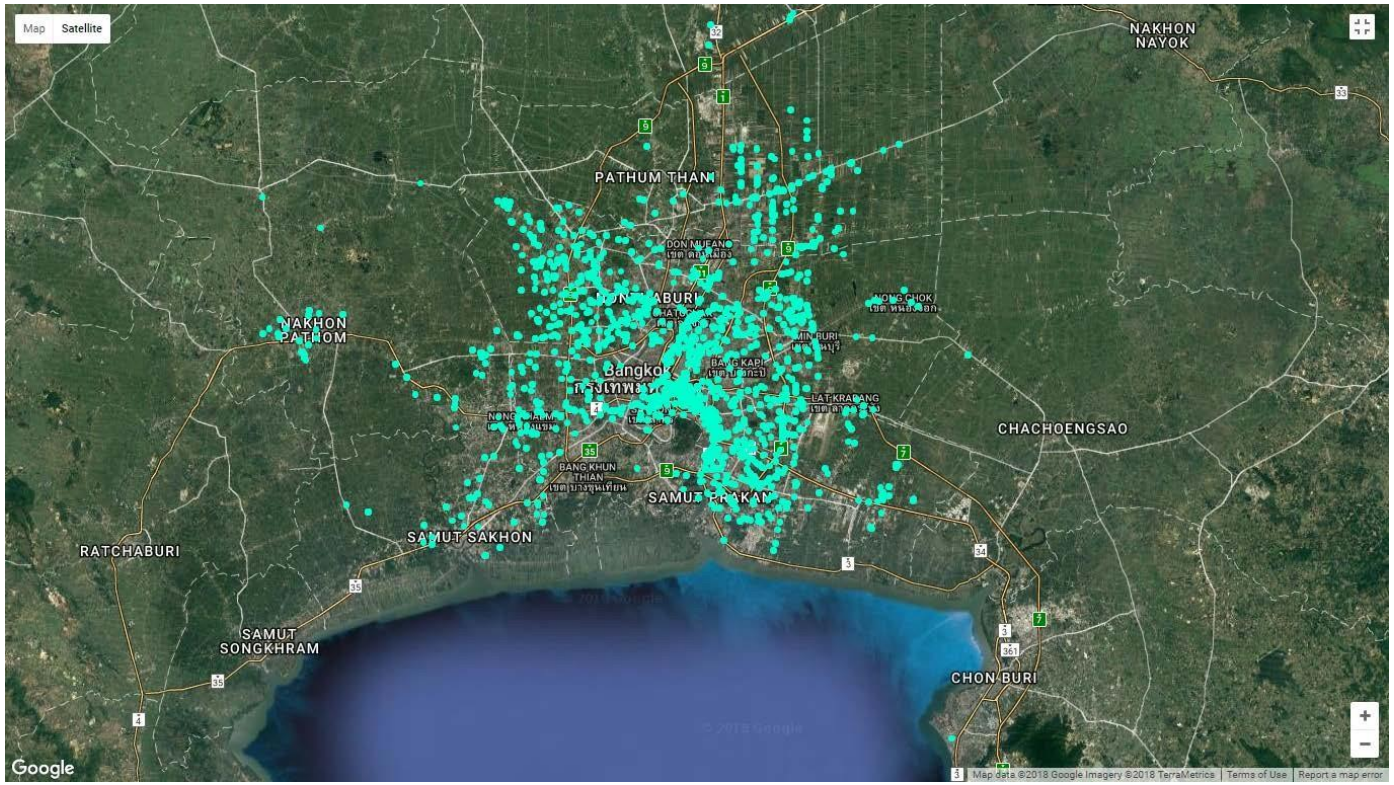


Example - Customer analysis



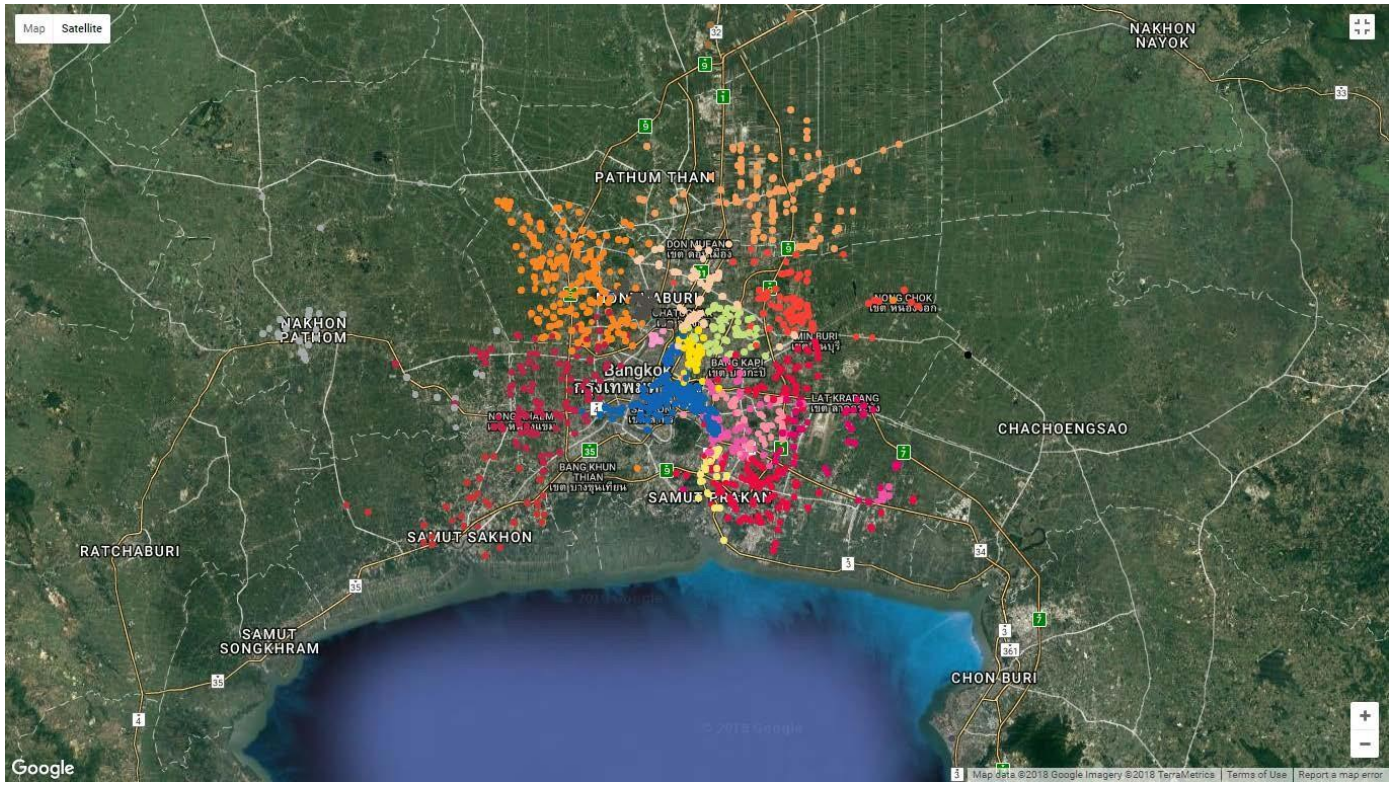
Example - Real Estate segmentation in Thailand

What should be the input feature of this?

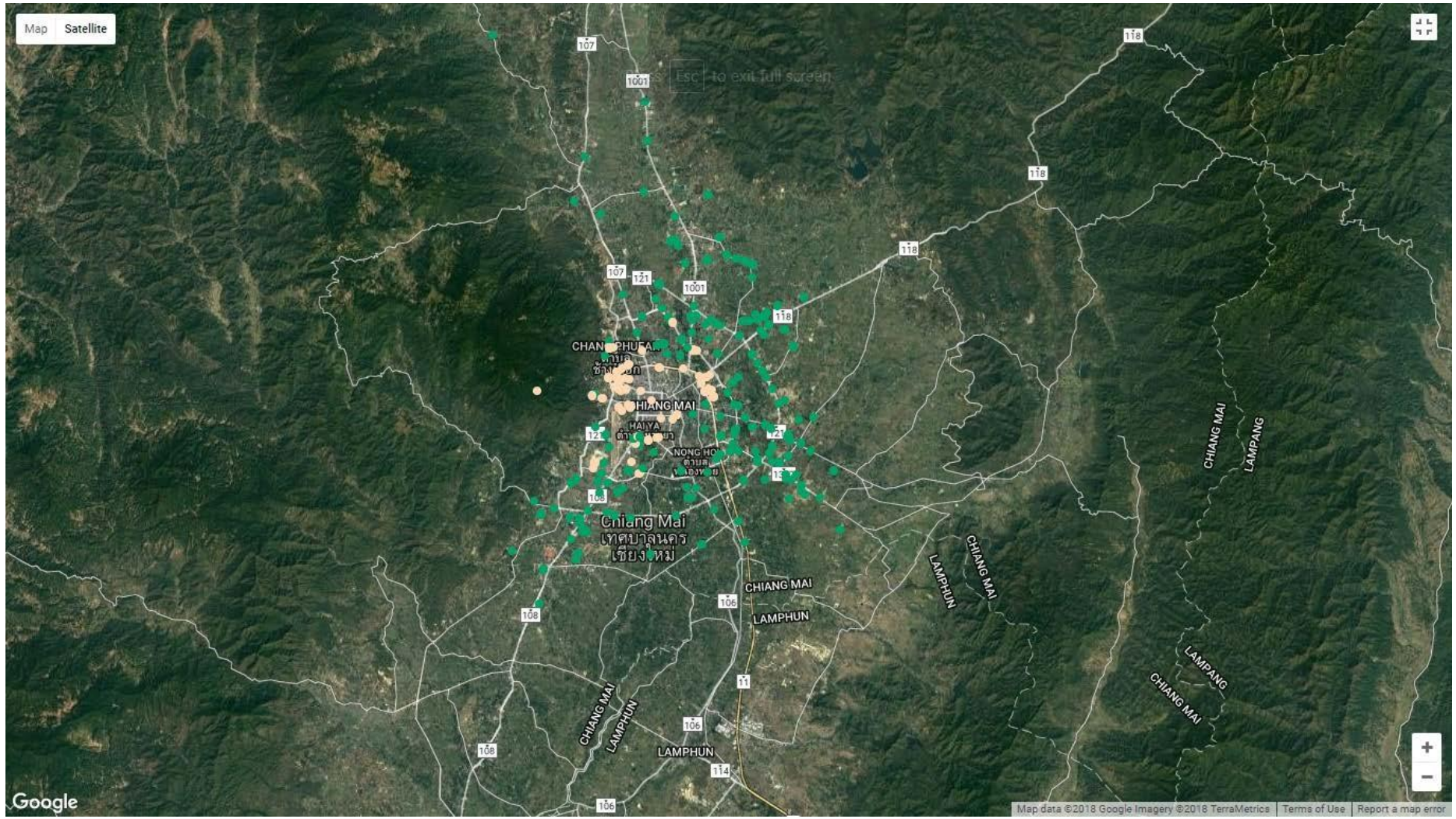


Example - Real Estate segmentation in Thailand

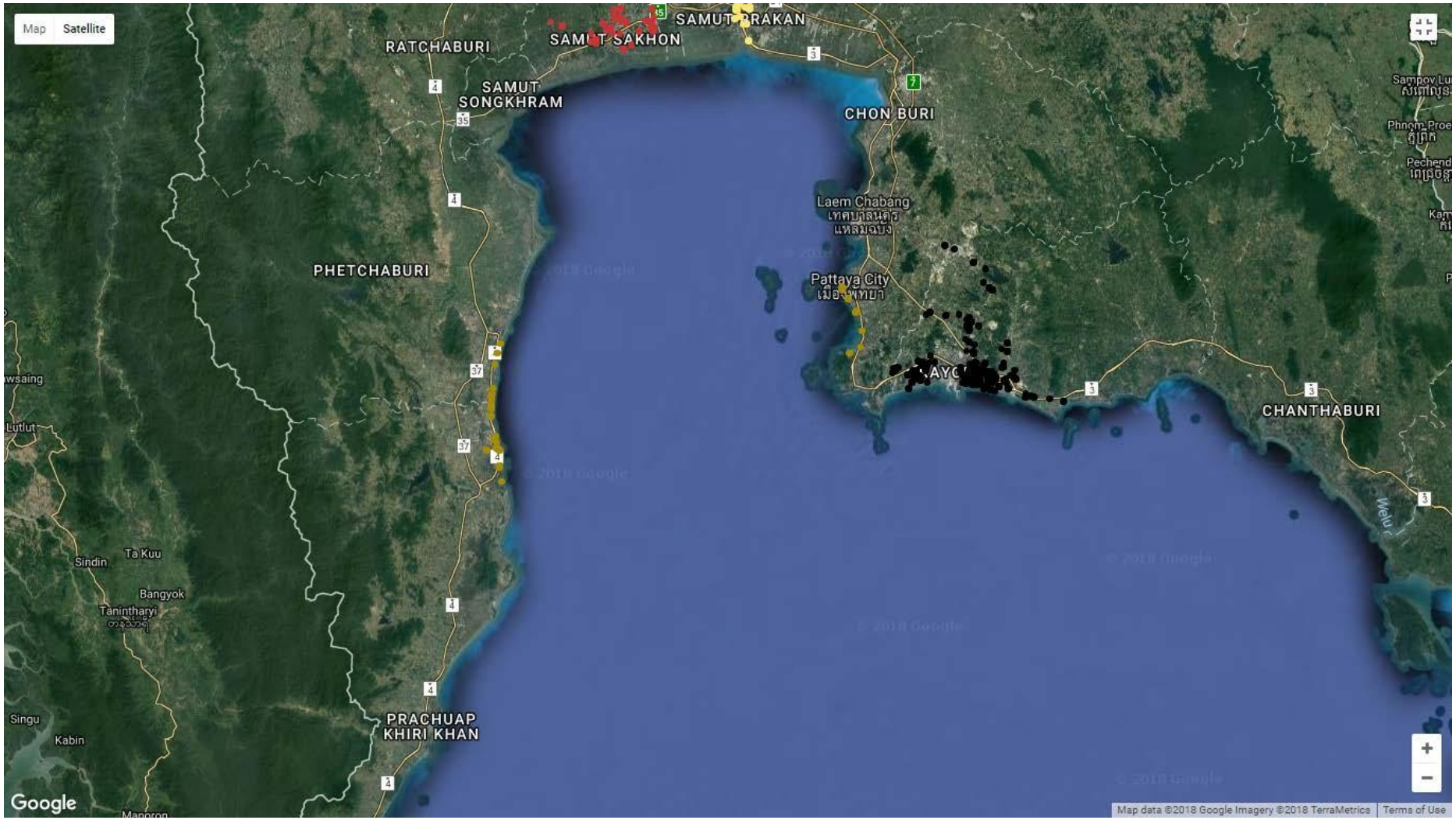
What should be the input feature of this?



Example - Real Estate segmentation in Thailand



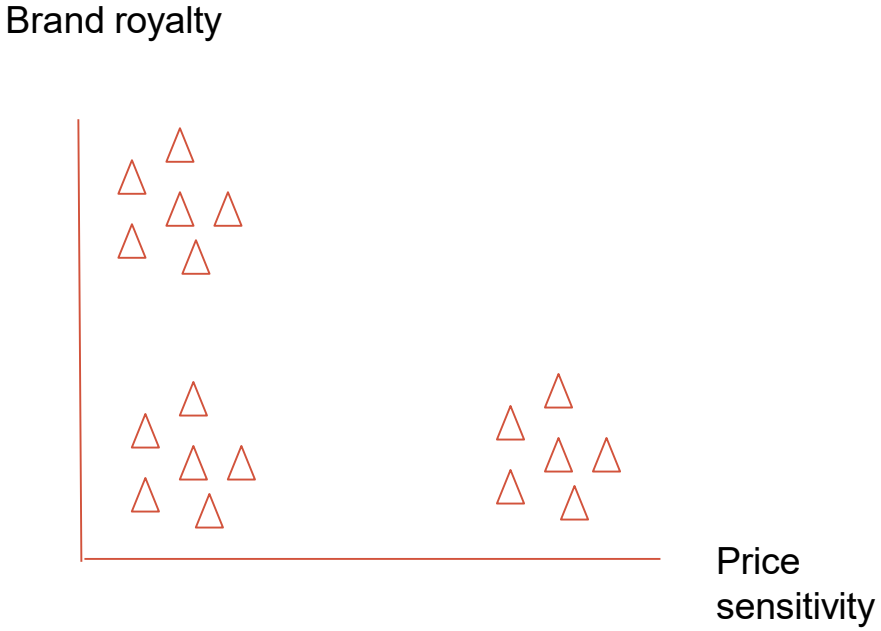
Example - Real Estate segmentation in Thailand



K-mean clustering

Clustering - task that tries to automatically discover groups within the data

Too hard…

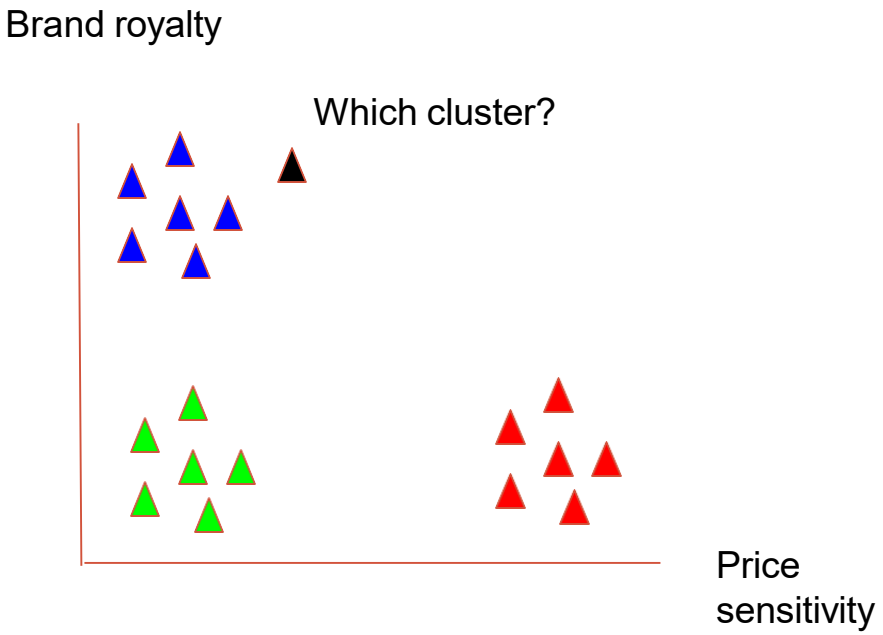


K-mean clustering

Clustering - task that tries to automatically discover groups within the data

Too hard…
Easier if we know the grouping beforehand (supervised)

How?



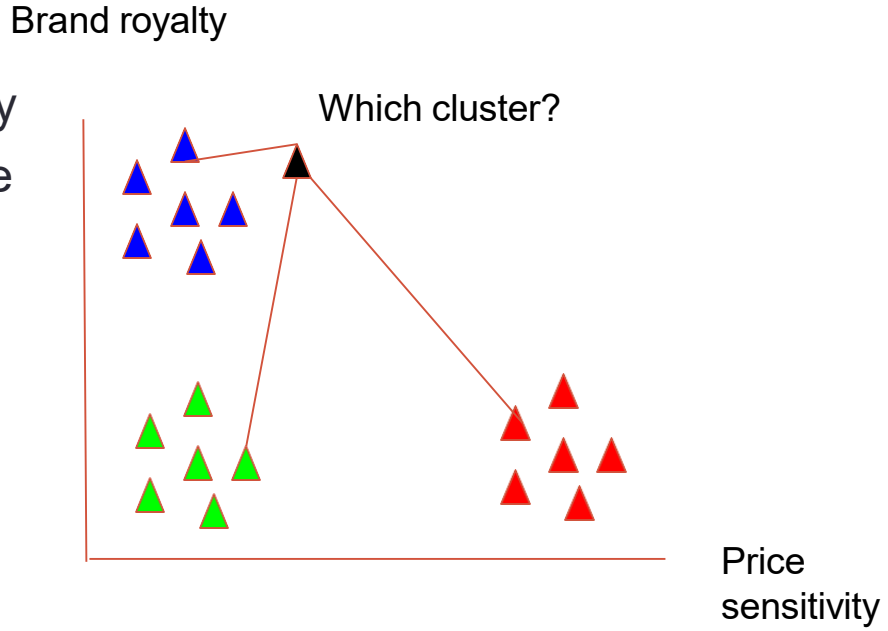
Nearest Neighbour classification

Find the closest training data, assign the same label as the training data

Given query data

For every point in the training data

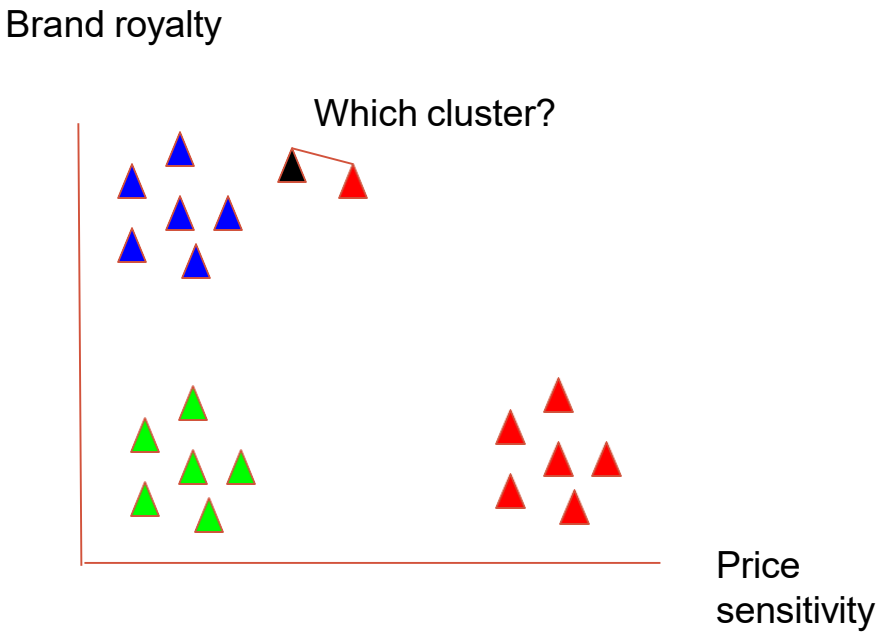
- Compute the distance with the query
- Assign label of the smallest distance



K-Nearest Neighbour (kNN) classification

Nearest Neighbour is susceptible to noise in the training data

Use a voting scheme instead



K-Nearest Neighbour (kNN) classification

Nearest Neighbour is susceptible to noise in the training data

Use a voting scheme instead

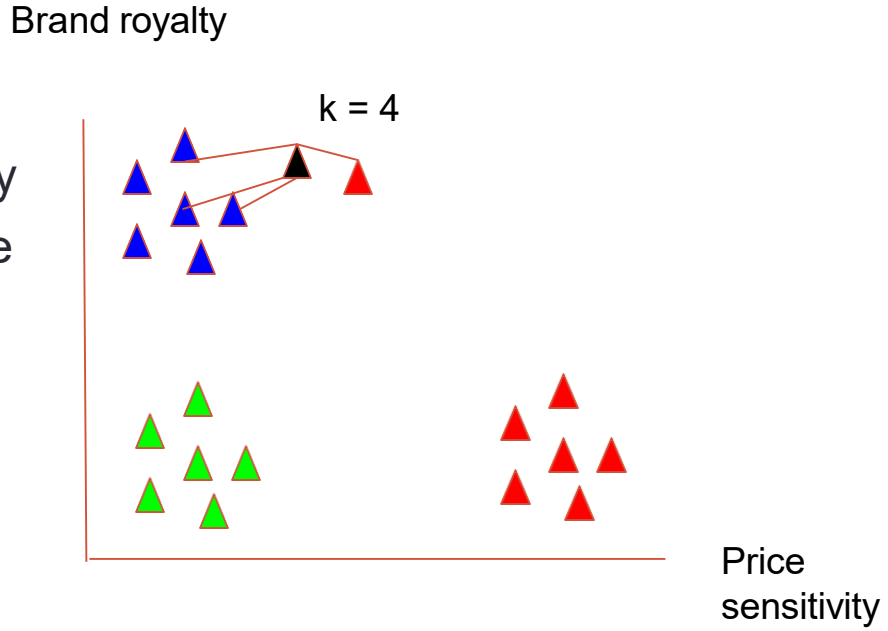
Given query data

For every point in the training data

Compute the distance with the query

Assign label of the smallest distance

Assign label by voting



The votes can be weighted by the inverse distance (weighted k-NN)

Closest?

We need some kind of distance or similarity measures

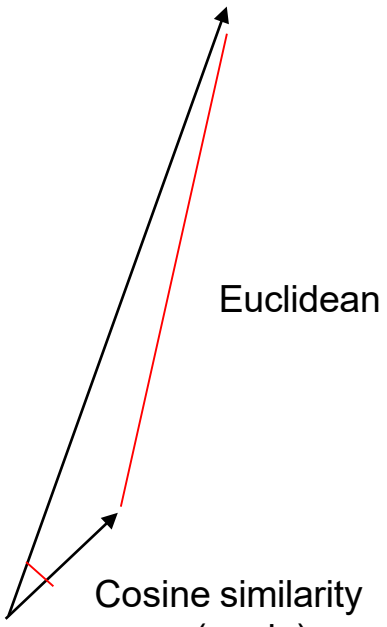
F(**X**$_1$,**X**$_2$) = d

$$X_1 = [x_{1,1}, x_{1,2}, \ldots, x_{1,n}]$$

$$X_2 = [x_{2,1}, x_{2,2}, \ldots, x_{2,n}]$$

Euclidean distance $$\sqrt{\Sigma_i (x_{1,i} - x_{2,i})^2}$$

Cosine similarity $$\frac{X_1 \cdot X_2}{|X_1||X_2|} = \frac{\Sigma_i x_{1,i} x_{2,i}}{\sqrt{\Sigma_i x_{1,i}^2}\sqrt{\Sigma_i x_{2,i}^2}}$$



Many more distances, Jaccard distance, Earth mover distance

KNN runtime

For every point in the training data

Compute the distance with the query

Find the K closest data points

Assign label by voting

O(N)

O(JN) - If we have J queries

Expensive

Ways to make it faster

Kernelized KNN

Locally Sensitive Hashing (LSH)

Use centroids

Centroids

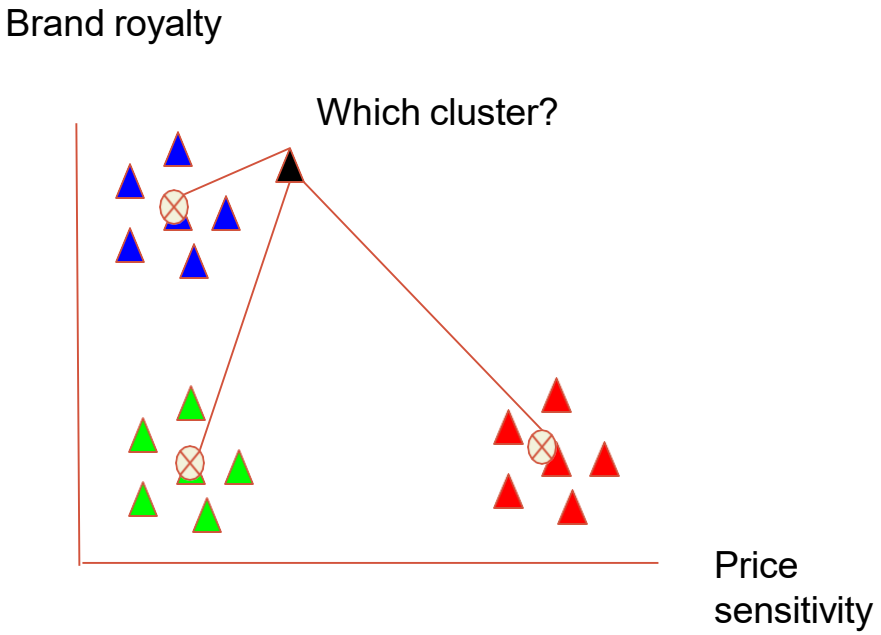
Basically, the representative of the cluster

Find the mean location of the cluster by averaging

Can use mode or median depending on the data

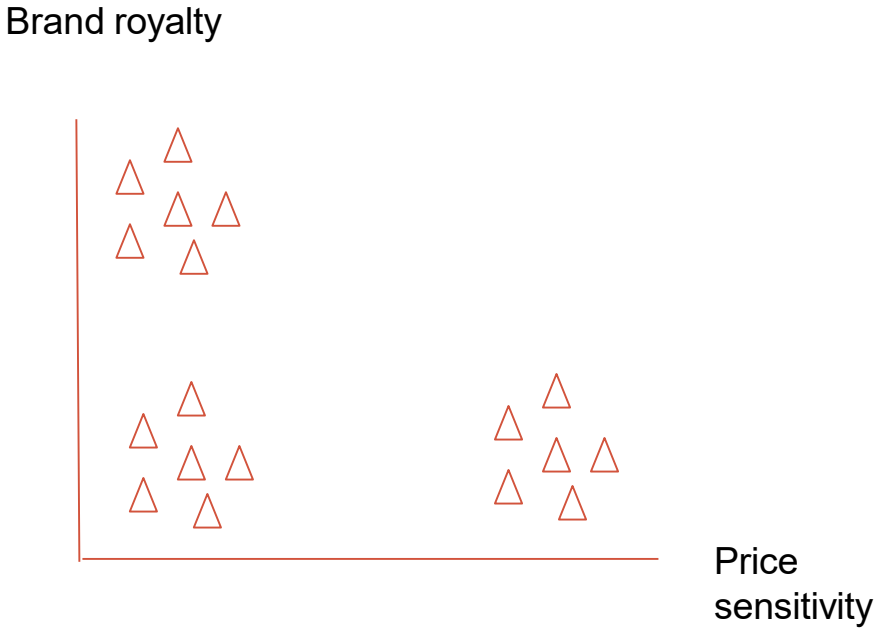
O(JL)

L - number of clusters

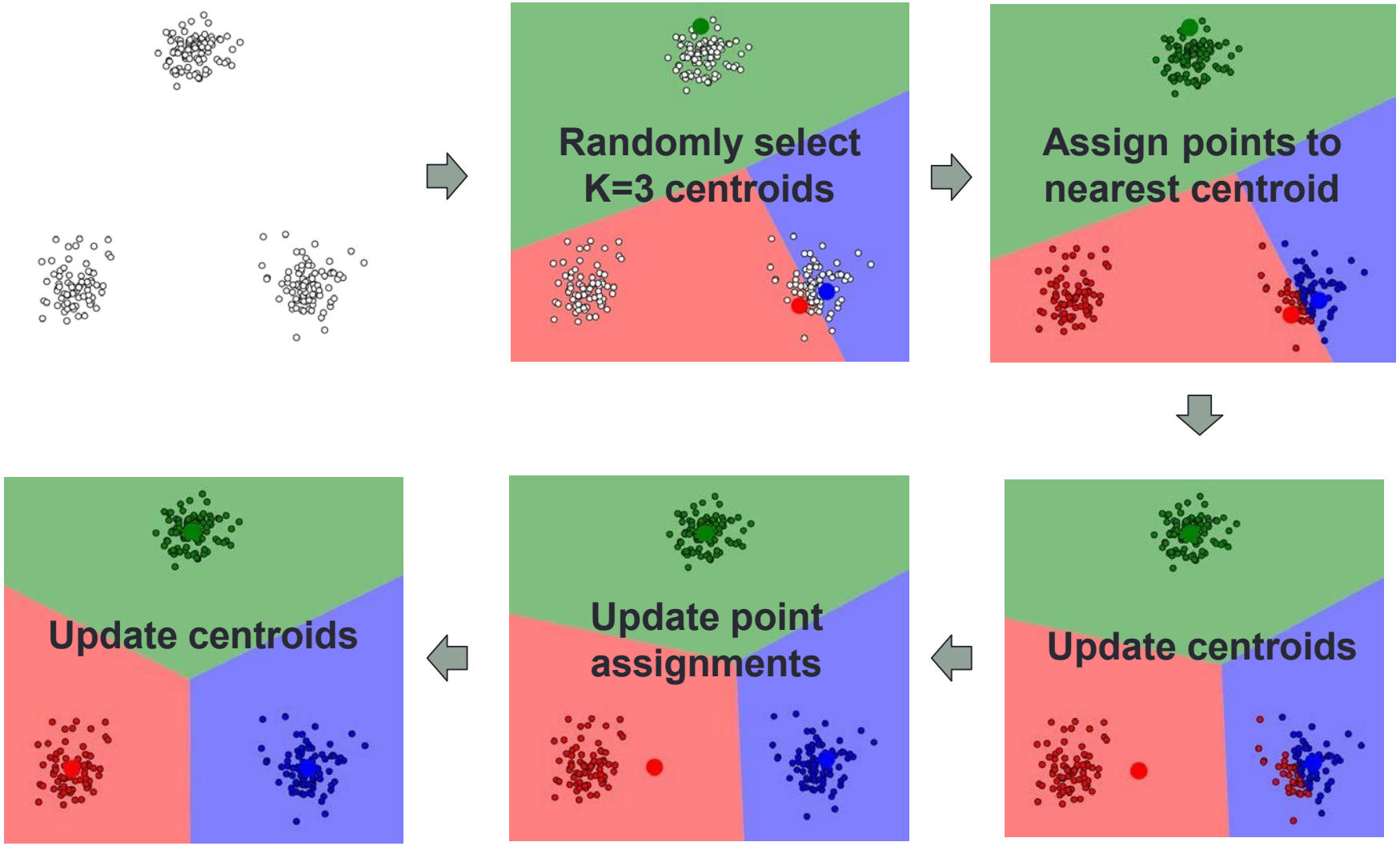


K-mean clustering

1. Randomly init k centroids by picking from data points
2. Assign each data points to centroids
3. Update centroids for each cluster
4. Repeat 2-3 until centroids does not change



An Illustration Of K-Mean Clustering



From https://www.naftaliharris.com/blog/visualizing-k-means-clustering/

Characteristics of K-means

- **The number of clusters, *K*, is specified in advance.**
- **Always converge to a (local) minimum.**
  - **Poor starting centroid locations can lead to incorrect minima.**

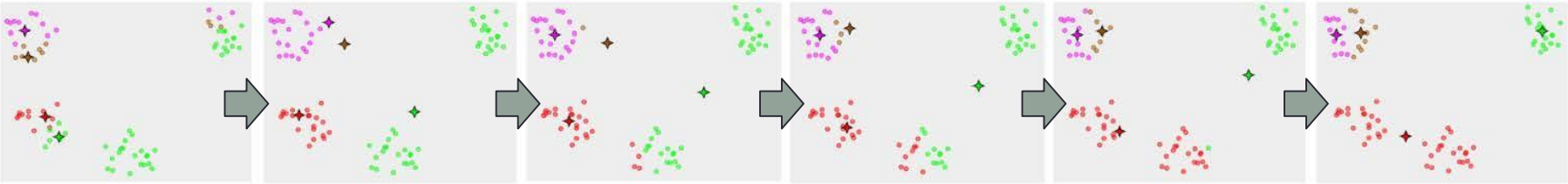
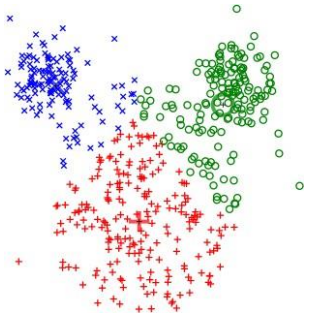
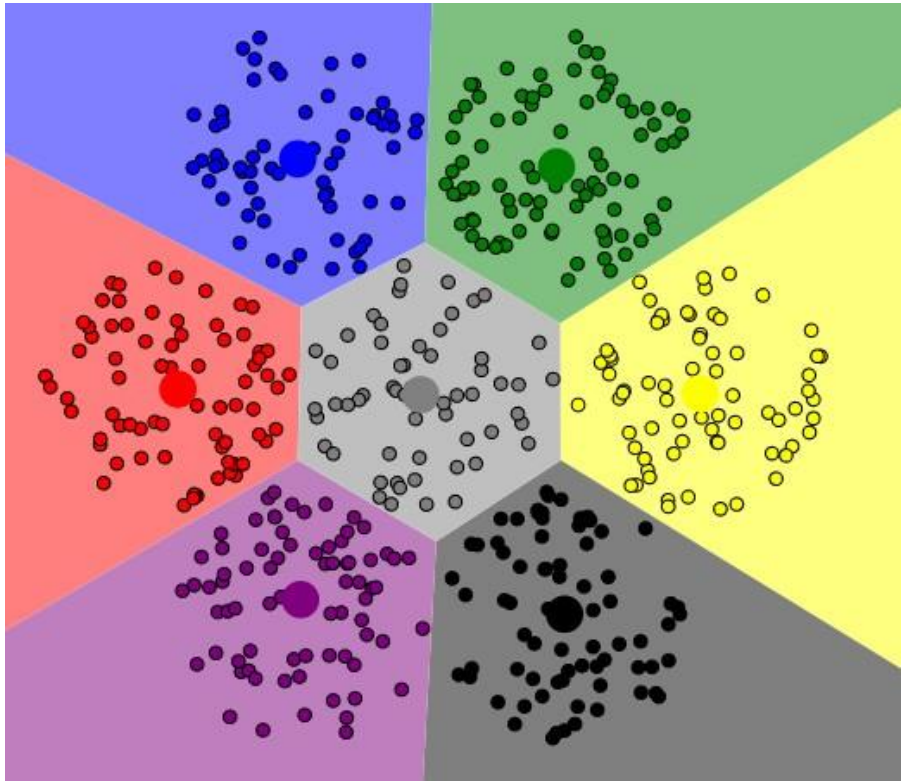


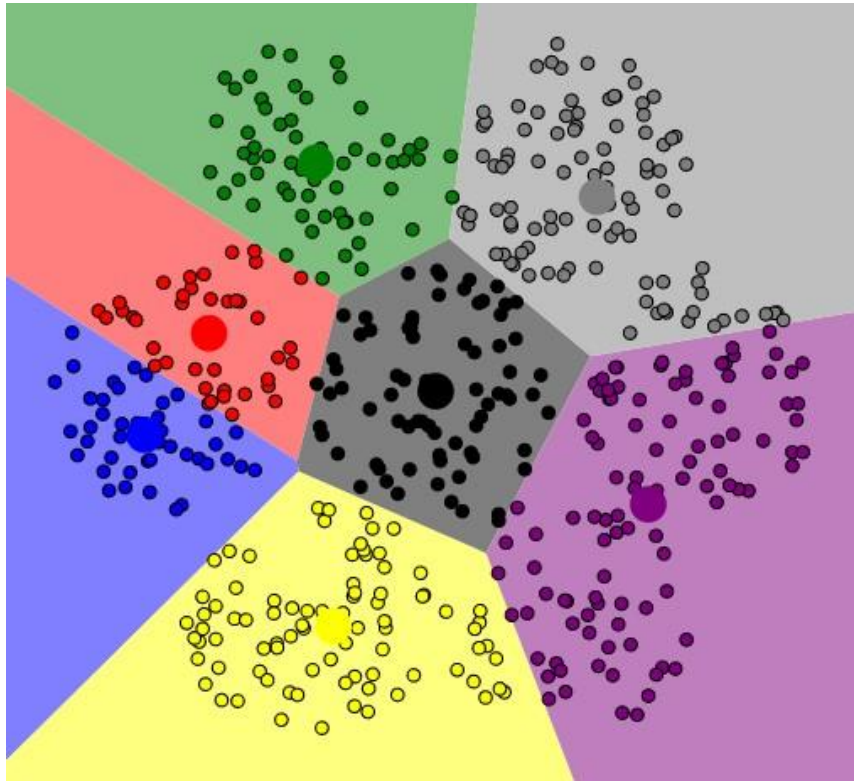
Image from https://en.wikipedia.org/wiki/K-means_clustering

- **The model has several implicit assumptions:**
  - **Data points scatter around cluster's centers.**
  - **Boundary between adjacent clusters is always halfway between the cluster centroids.**



Effect of bad initializations





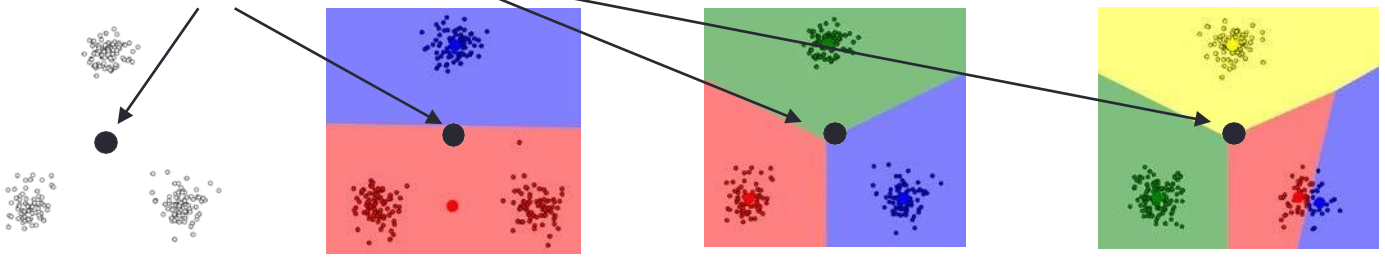
Solution, try different randomization and pick the best

From https://www.naftaliharris.com/blog/visualizing-k-means-clustering/

Selecting K - Using Elbow method

All-data centroid

From https://www.naftaliharris.com/blog/visualizing-k-means-clustering/



K = 1 **K = 2** **K = 3** **K = 4** ...

fraction of explained variance = $\dfrac{\textbf{between-cluster variance}}{\textbf{all-data variance}}$

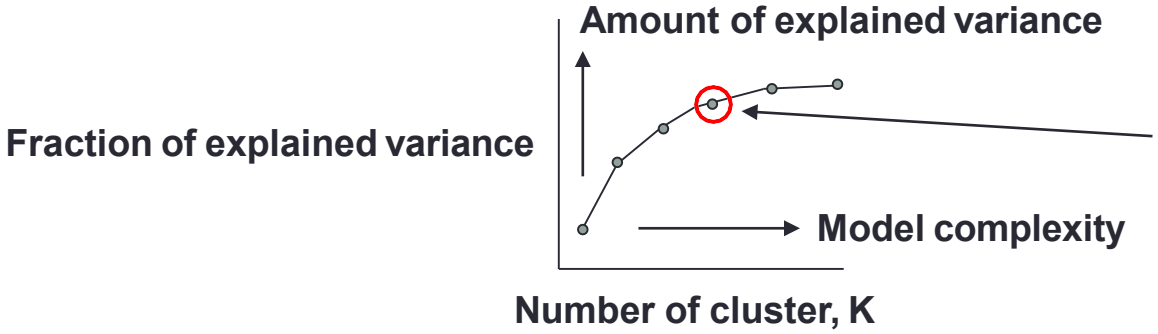
(note: this is Euclidean distance)

between-cluster variance = $\sum_{i=1}^{K} \frac{n_i (M_i - M)^2}{N-1}$, where n_i = size of ith cluster,

M_i = centroid of ith cluster, and

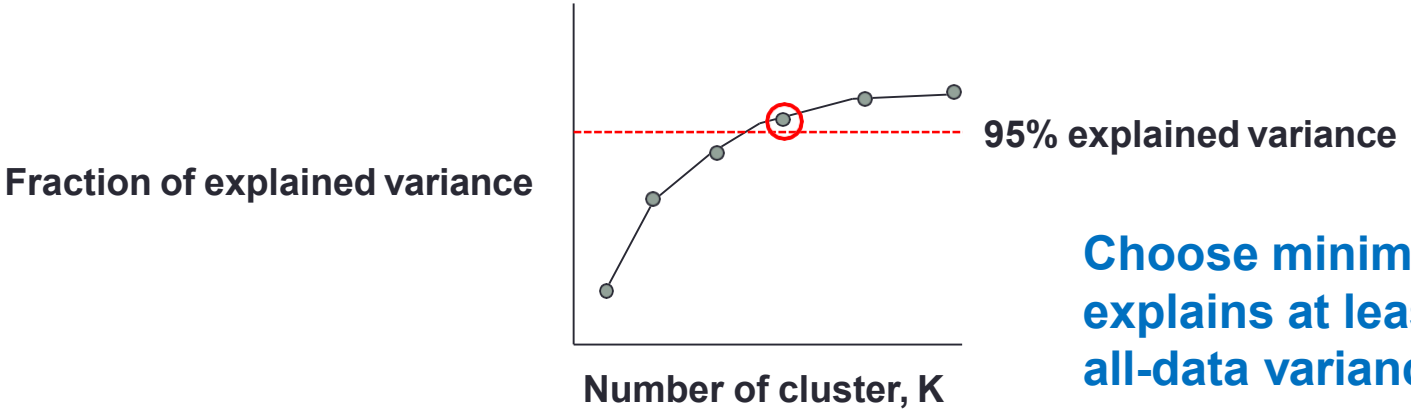
M = all-data centroid.

all-data variance = $\sum_{i=1}^{N} \frac{(x_i - M)^2}{N-1}$, where x_i = i^{th} data point and N = # of data.



The elbow method chooses K where increasing complexity doesn't yield much in return.

Selecting K - other methods



Choose minimal K that explains at least 95% of the all-data variance.

K = 2

K = 3

K = 4

⋮

→ Training K-mean Clustering Model → Testing / Cross-validation →

K	Accuracy
2	50%
3	68%
4	83%
⋮	⋮

Choose K that maximizes certain objective (e.g. accuracy on testing data)

Best method

REGRESSION

with some K-mean clustering

Predicting amount of rainfall

https://esan108.com/%E0%B8%9E%E0%B8%A3%E0%B8%B0%E0%B9%82%E0%B8%84%E0%B8%81%E0%B8%B4%E0%B8%99%E0%B8%AD%E0%B8%B0%E0%B9%84%E0%B8%A3%E0%B8%AB%E0%B8%A1%E0%B8%B2%E0%B8%A2%E0%B8%96%E0%B8%B6%E0%B8%87.html

Predicting amount of rainfall

Cloth	Corn	Grass	Water	Beer	Rainfall
4	6	3	10	0	**76950**
5	1	0	0	7	**30234**
6	0	3	5	7	**123456**
5	0	3	12	0	**89301**
4	3	0	6	7	**?**

We assume the input features have some correlation with the amount of rainfall.

Can we create a model that predict the amount of rainfall?

What is the output?

What is the input (features)?

Predicting the amount of rainfall

- The correlation can be positive or negative

Predicting the amount of rainfall

Cloth	Corn	Grass	Water	Beer	Rainfall
4	6	3	10	0	**76950**
5	1	0	0	7	**30234**
6	0	3	5	7	**123456**
5	0	3	12	0	**89301**
4	3	0	6	7	**?**

- $h_{\boldsymbol{\theta}}(\mathbf{x_1}) = \theta_0 + \theta_1 x_{1,1} + \theta_2 x_{1,2} + \theta_3 x_{1,3} + \theta_4 x_{1,4} + \theta_5 x_{1,5}$

  1 refers to index of the data (the first in the training/test set)
- Where θs are the parameter of the model
- Xs are values in the table

(Linear) Regression

- $h_{\boldsymbol{\theta}}(\mathbf{x_1}) = \theta_0 + \theta_1 x_{1,1} + \theta_2 x_{1,2} + \theta_3 x_{1,3} + \theta_4 x_{1,4} + \theta_5 x_{1,5}$
- θs are the parameter (or weights)

Assume x_0 is always 1

- We can rewrite

n is dimension of x

$$h_\theta(\mathbf{x}_i) = \Sigma_{j=0}^{n} \theta_j x_{i,j} = \theta^T \mathbf{x}_i$$

h is parameterized by θ

- Notation: vectors are bolded
- Notation: vectors are column vectors

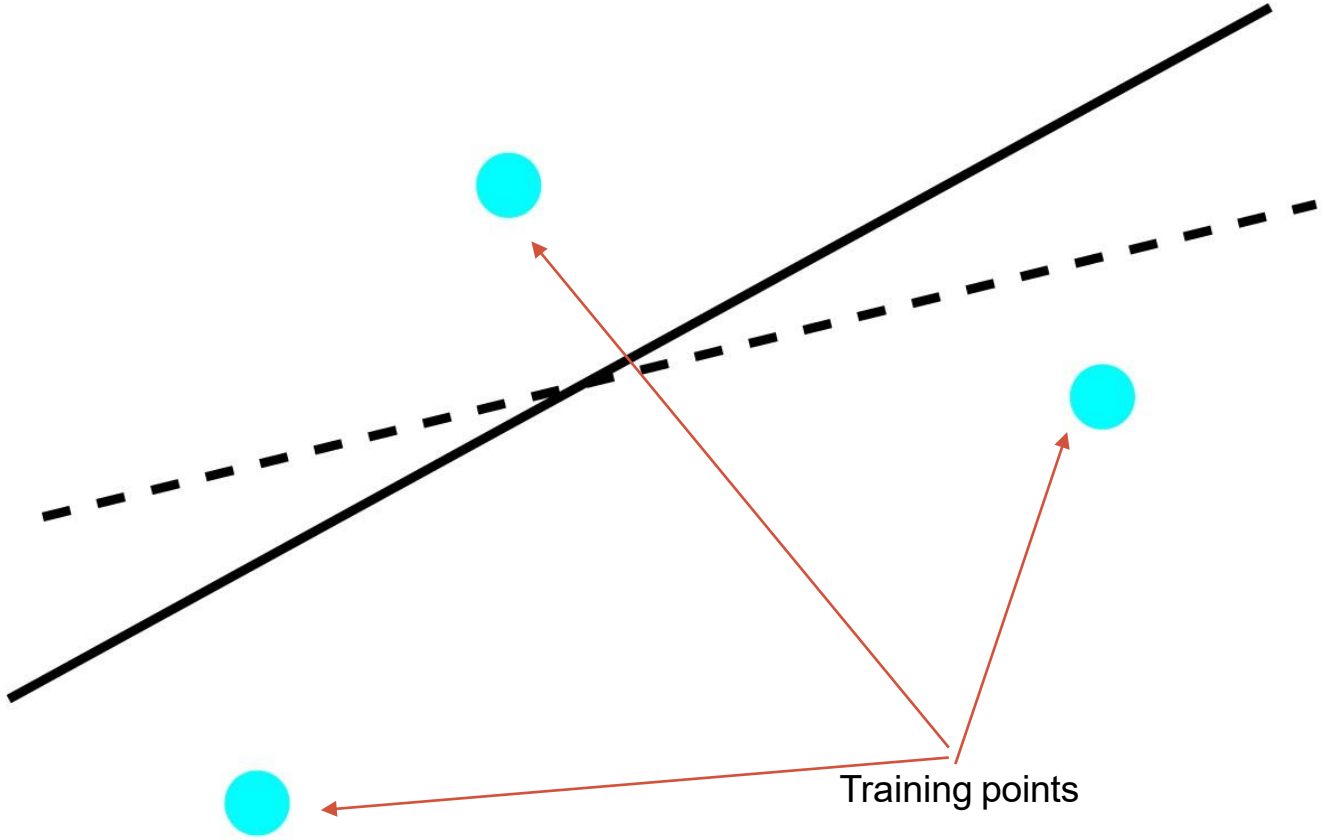
Picking θ

- Random until you get the best performance?

- How to quantify best performance?

$$h_\theta(\mathbf{x}_i) = \Sigma_{j=0}^{n} \theta_j x_{i,j} = \theta^T \mathbf{x}_i$$

$$h_\theta(\mathbf{x}_i) = \Sigma_{j=0}^{n} \theta_j x_{i,j} = \theta^T \mathbf{x}_i$$



https://chem.libretexts.org/Textbook_Maps/Analytical_Chemistry_Textbook_Maps/Map%3A_Analytical_Chemistry_2.0_(Harvey)/05_Standardizing_Analytical_Methods/5.4%3A_Linear_Regression_and_Calibration_Curves

Cost function (Loss function)

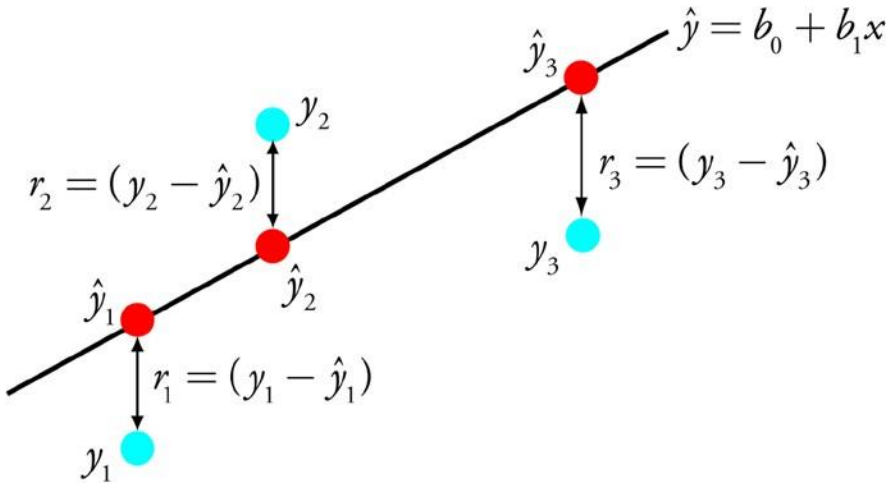
- Let's use the mean square error (MSE)

m is the number of training examples

$$J(\theta) = \frac{1}{m}\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x_i})^2$$

i here is the index of the training example

We want to pick **θ** that minimize the loss



Cost function (Loss function)

- Let's use the mean square error (MSE)

$$J(\theta) = \frac{1}{m}\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x_i})^2$$

We want to pick **θ** that minimize the loss

$$\frac{m}{2}J(\theta) = \boxed{\frac{1}{2}\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x_i})^2}$$

Picking θ

- Random until you get the best performance?
  - Can we do better than random chance?

- How to quantify best performance?

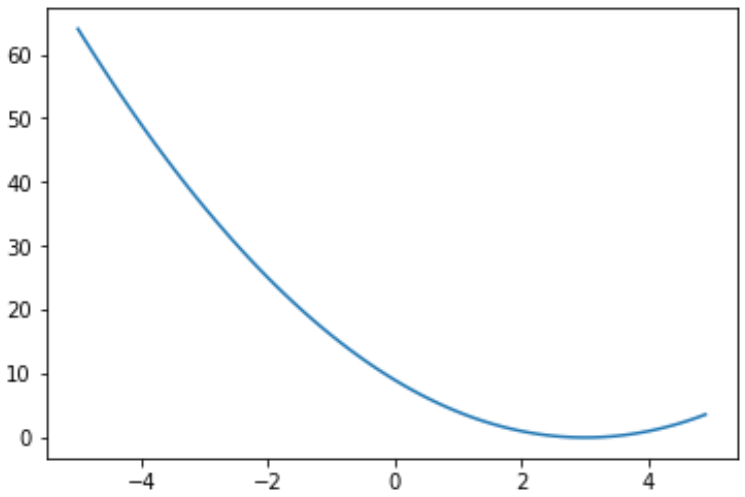
$$\frac{m}{2}J(\theta) = \frac{1}{2}\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x_i})^2$$

Minimizing a function

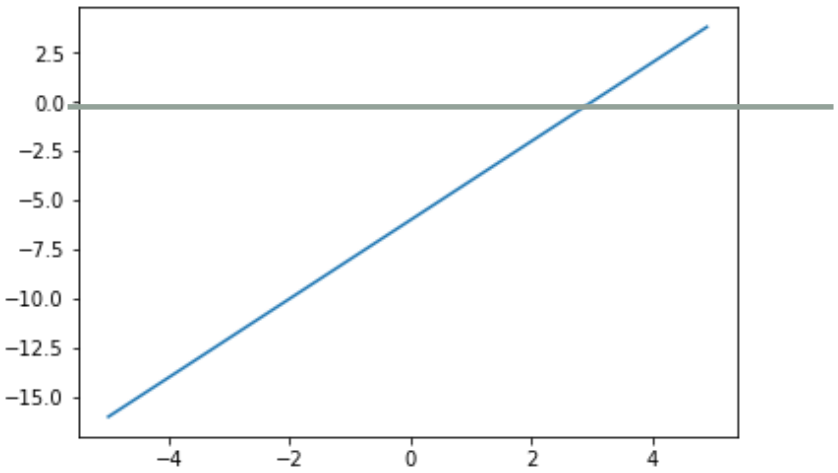
- You have a function
  - $y = (x - a)^2$
- You want to minimize Y with respect to x
  - $dy/dx = 2x - 2a$
  - Take the derivative and set the derivative to 0
    - (And maybe check if it's a minima, maxima or saddle point)

- We can also go with an iterative approach

Gradient descent



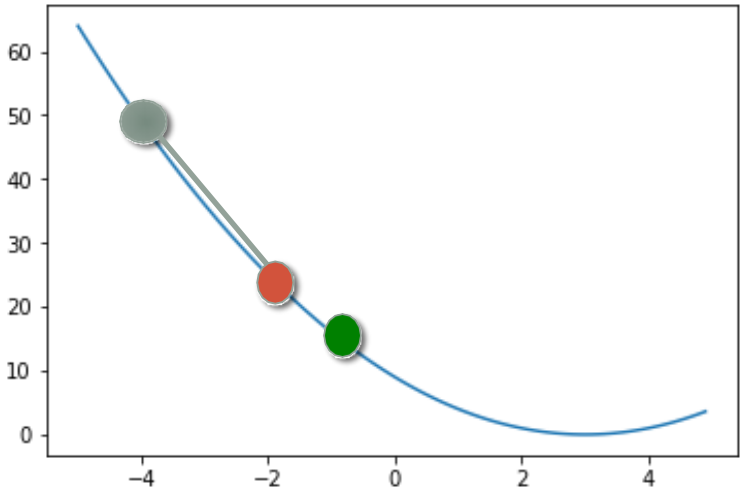
y



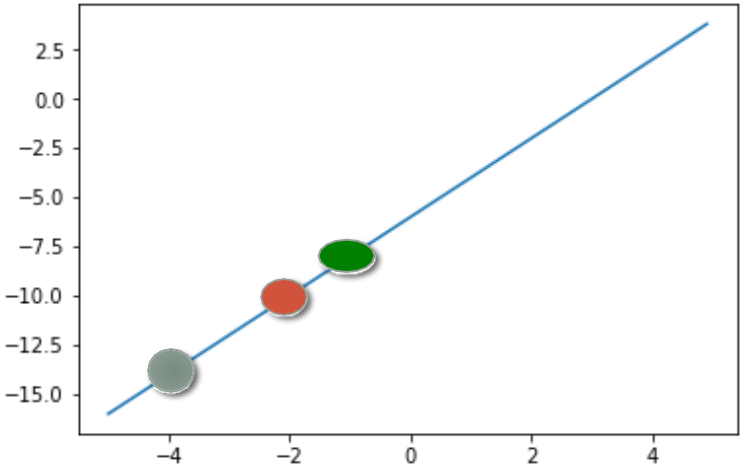
dy/dx

First what does dy/dx means?

Gradient descent



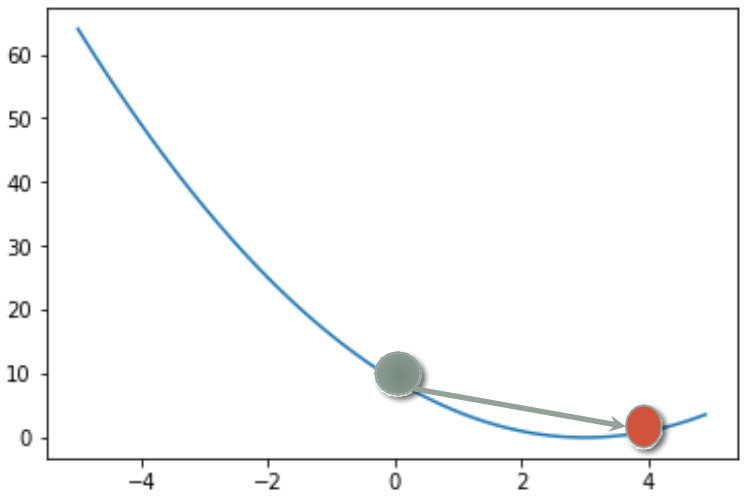
y



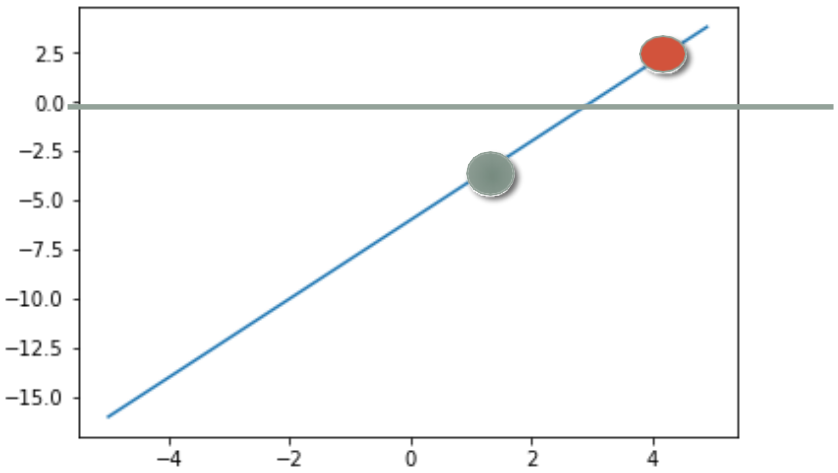
dy/dx

Move along the negative direction of the gradient
The bigger the gradient the bigger step you move

Gradient descent



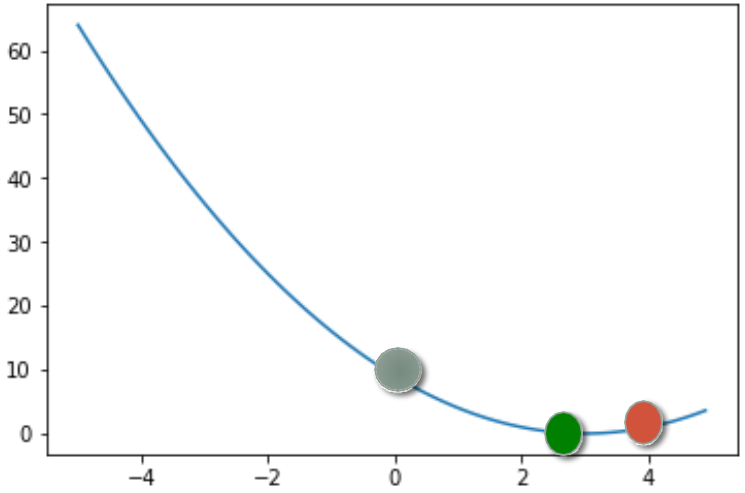
y



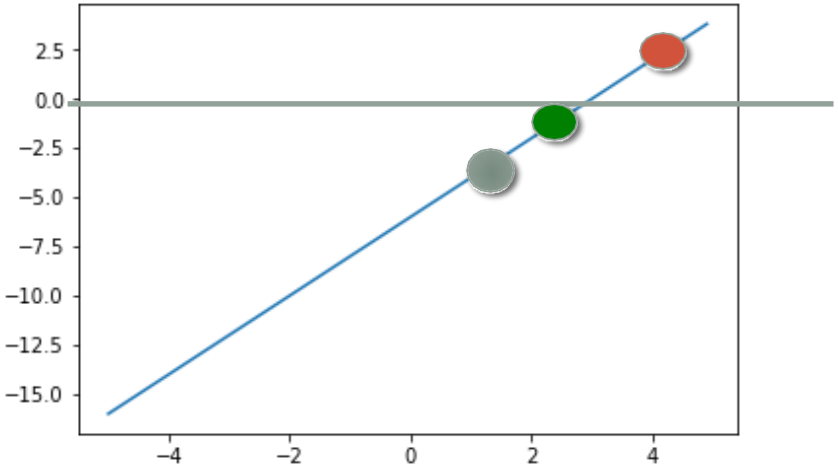
dy/dx

What happens when you overstep?

Gradient descent



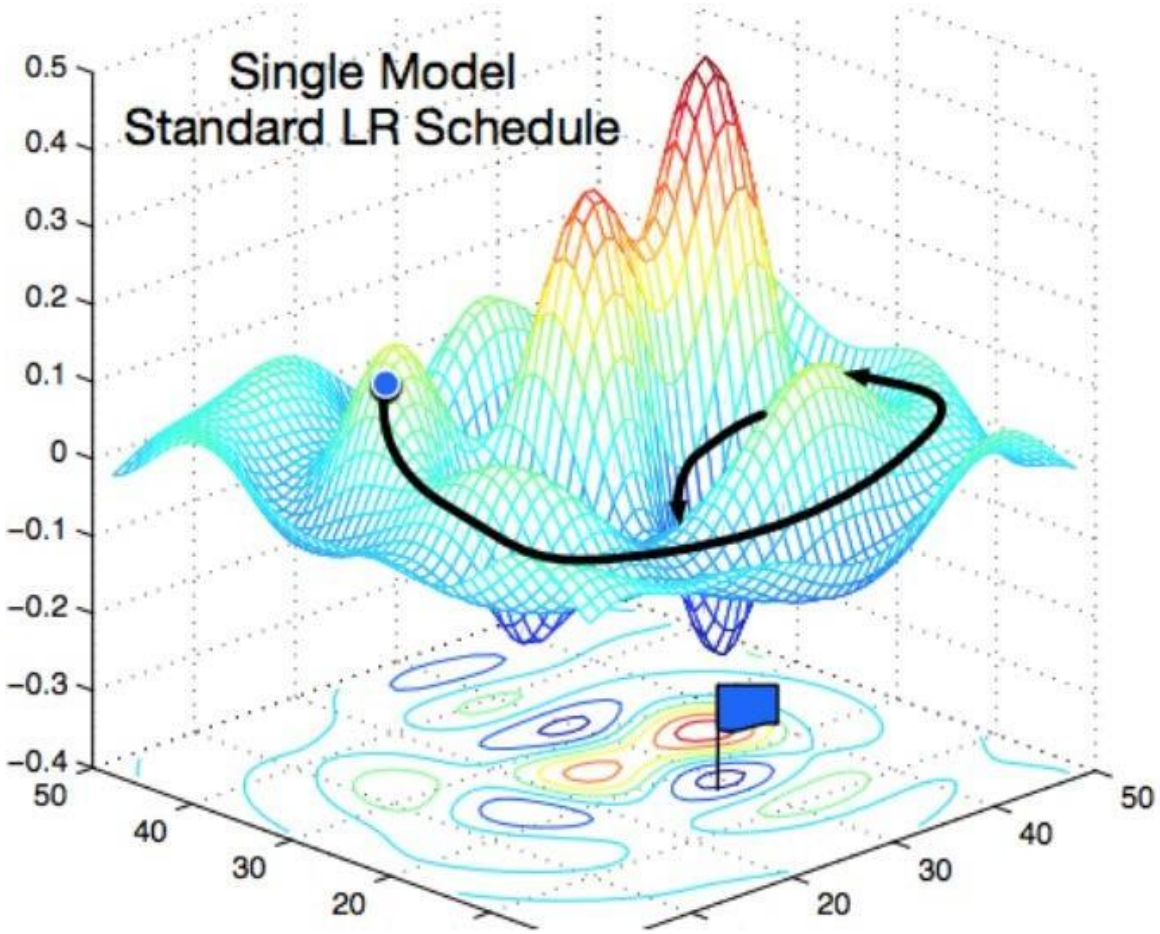
y



dy/dx

If you over step you can move back

Gradient descent in 3d



https://openreview.net/pdf?id=BJYwwY9ll

Formal definition

- $y = f(x)$

- Pick a starting point x_0

- Moves along -dy/dx
- $x_{n+1} = x_n – r * dy/dx$
- Repeat till convergence
- r is the learning rate

  Big r means you might overstep

  Small r and you need to take more steps

Picking θ

- Random until you get the best performance?
  - Can we do better than random chance?
    - Gradient descent (a better guess!)

- How to quantify best performance?

$$\frac{m}{2}J(\theta) = \frac{1}{2}\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x_i})^2$$

LMS regression with gradient descent

$$J(\theta) = \frac{1}{2}\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x_i})^2$$

Gradient

$$\frac{\partial J}{\partial \theta_j} = -\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x}_i)x_i^{(j)}$$

$$J(\theta) = \frac{1}{2}\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x_i})^2$$

LMS regression with gradient descent

$$\frac{\partial J}{\partial \theta_j} = -\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x}_i)x_i^{(j)}$$

Update rule for linear regression

$$\theta_j \Leftarrow \theta_j + r\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x}_i)x_i^{(j)}$$

Interpretation?

Batch updates vs mini-batch

$$\theta_j \Leftarrow \theta_j + r\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x}_i)x_i^{(j)}$$

- Batch updates (considering the whole training data) estimate the Loss function precisely
  - Can takes a long time if m is large
- Updates with a subset of m
  - We now have an estimate of the loss function
    - This can lead to a wrong direction, but we get faster updates
  - Called Stochastic Gradient Descent (SGD) or incremental gradient descent

Minimizing a function

- You have a function
  - $y = (x - a)^2$
- You want to minimize Y with respect to x
  - $dy/dx = 2x - 2a$
  - Take the derivative and set the derivative to 0
    - (And maybe check if it's a minima, maxima or saddle point)

- We can also go with an iterative approach (Gradient descent)

LMS regression with matrix derivatives

- First let's definite what's a derivative of a matrix
- For a function $f : \mathbb{R}^{m \times n} \mapsto \mathbb{R}$

- The derivative wrt to A is

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial A_{m1}} & \cdots & \frac{\partial f}{\partial A_{mn}} \end{bmatrix}$$

http://cs229.stanford.edu/notes/cs229-notes1.pdf

Example

- Suppose

$$f : \mathbb{R}^{m \times n} \mapsto \mathbb{R}$$

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \qquad \nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial A_{m1}} & \cdots & \frac{\partial f}{\partial A_{mn}} \end{bmatrix}$$

$$f(A) = \frac{3}{2} A_{11} + 5A_{12}^2 + A_{21} A_{22}$$

$$\nabla_A f(A) = \begin{bmatrix} \frac{3}{2} & 10A_{12} \\ A_{22} & A_{21} \end{bmatrix}$$

Trace of a matrix

- trA is the sum of the diagonals of matrix A (A must be a square matrix)

$$trA = \Sigma_i^N A_{ii}$$

- Trace of a real number? (1x1 matrix)

Trace properties

- tr (a) = a
- trA = trA^T
- tr(A+B) = trA+trB
- tr(aA) = atr(A)

$$\nabla_A trAB = B^T \tag{1}$$

$$\nabla_{A^T} f(A) = (\nabla_A f(A))^T \tag{2}$$

$$\nabla_A trABA^TC = CAB + C^TAB^T \tag{3}$$

$$\nabla_{A^T} trABA^TC = B^TA^TC^T + BA^TC \tag{4}$$

LMS regression with matrix derivatives

$$X = \begin{bmatrix} - & x_1^T & - \\ - & x_2^T & - \\ - & x_m^T & - \end{bmatrix} \qquad y = \begin{bmatrix} y_1 \\ y_2 \\ y_m \end{bmatrix}$$

$$X\theta - y = \begin{bmatrix} x_1^T\theta \\ | \\ x_m^T\theta \end{bmatrix} - \begin{bmatrix} y_1 \\ | \\ y_m \end{bmatrix}$$

LMS regression with matrix derivatives

$$X\theta - y = \begin{bmatrix} x_1^T\theta \\ | \\ x_m^T\theta \end{bmatrix} - \begin{bmatrix} y_1 \\ | \\ y_m \end{bmatrix}$$

$$\frac{1}{2}(X\theta - y)^T(X\theta - y) = \frac{1}{2}\sum_{i=1}^{m}(y_i - \theta^T x)^2$$

We want to minimize this term wrt to **θ**

$$\frac{1}{2}(X\theta - y)^T(X\theta - y) = \frac{1}{2}\sum_{i=1}^{m}(y_i - \theta^T x)^2$$

$$\theta = (X^T X)^{-1} X^T y$$

$$\theta = (X^T X)^{-1} X^T y$$

Other loss functions

- MSE

$$J(\theta) = \frac{1}{m}\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x_i})^2$$

  - Also called L2 loss
- L1 loss

$$\frac{1}{m}\Sigma_{i=1}^{m}|y_i - \theta^T \mathbf{x_i}|$$

Norms (p-norm or Lp-norm)

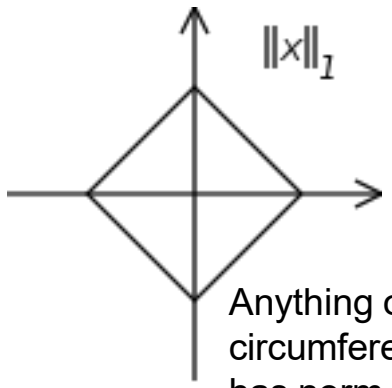
- For any real number p > 1

$$||\mathbf{x}||_p = (|x_1|^p + |x_2|^p + ... + |x_n|^p)^{\frac{1}{p}}$$

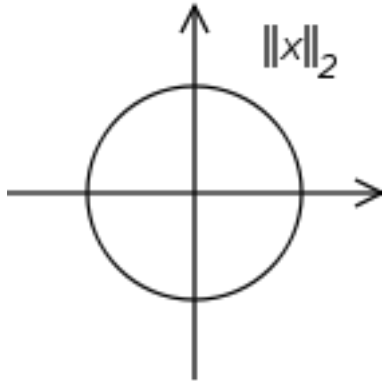
- For p = ∞

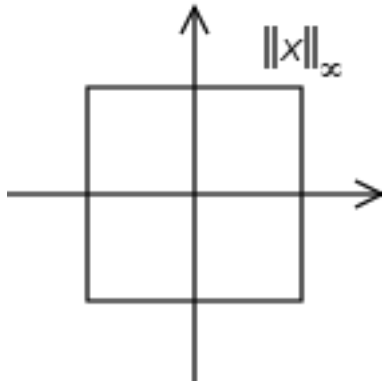
$$\|x\|_\infty = \max\{|x_1|, |x_2|, \ldots, |x_n|\}$$

- We'll see more of p-norms when we get to neural networks



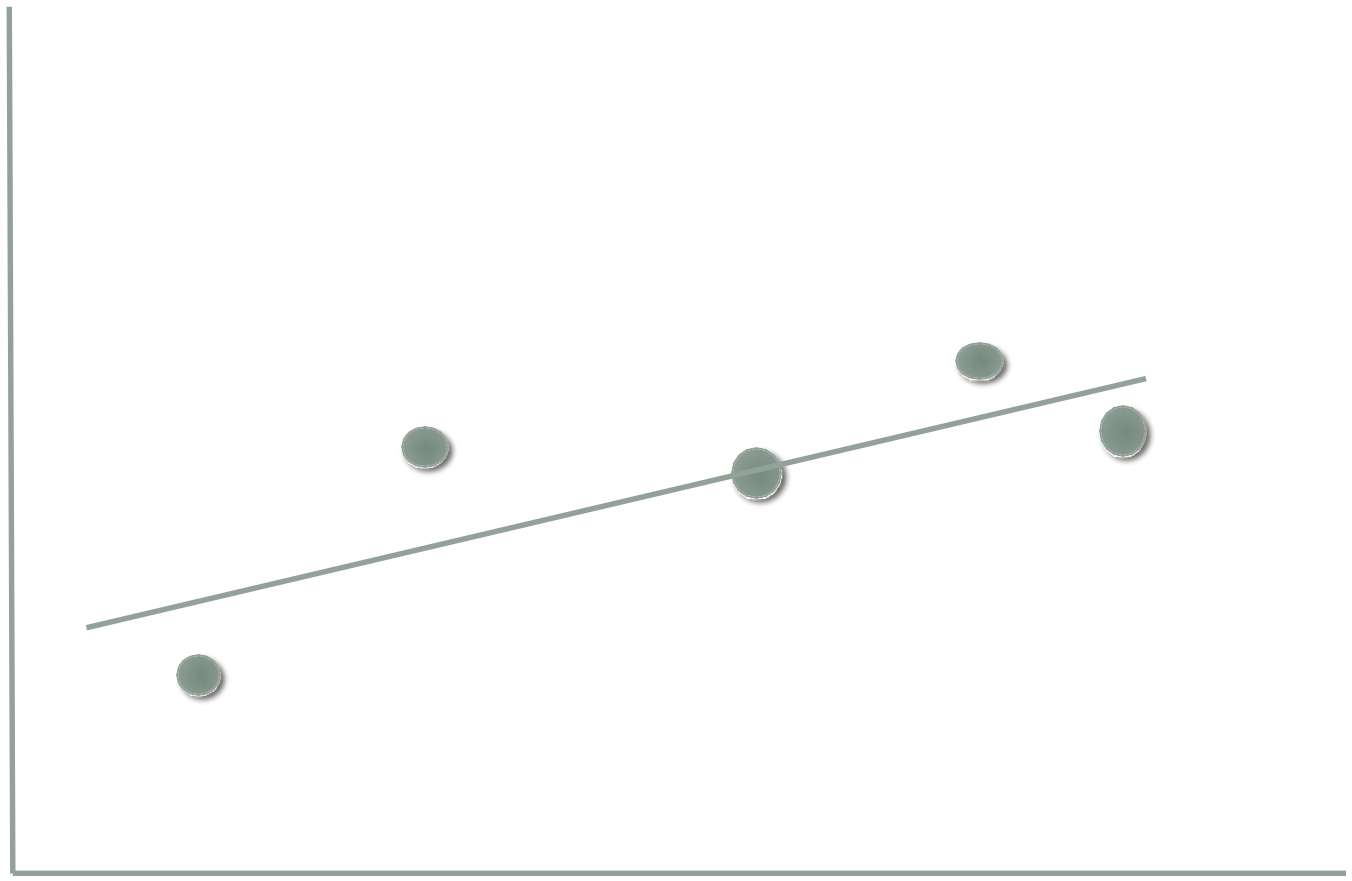
Anything on the circumference has norm of 1





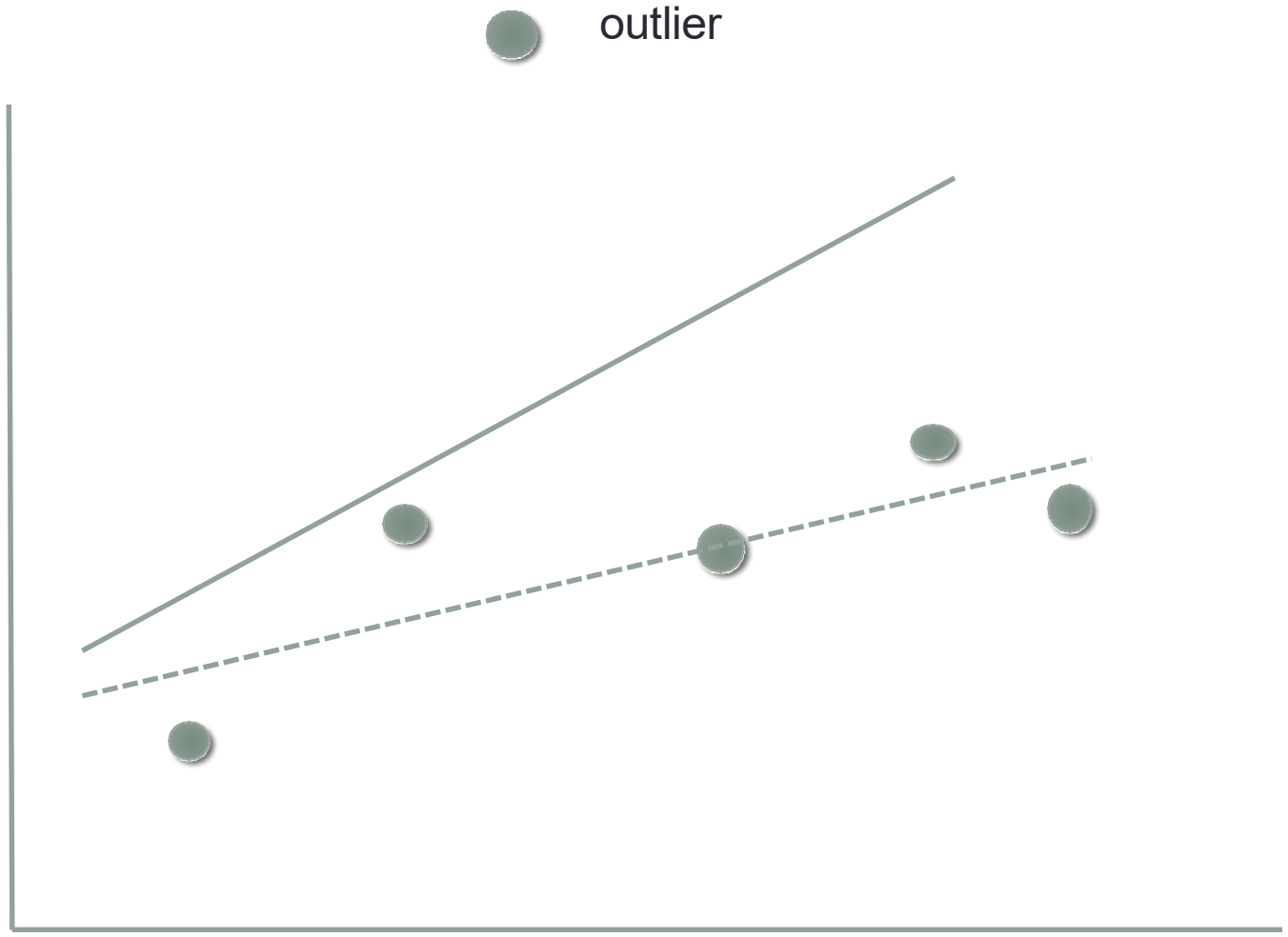
https://en.wikipedia.org/wiki/Lp_space

L2 vs L1 loss



L2 vs L1 loss

L2 is easier to find a solution compared to L1 (algorithmically)
To solve using L1 loss you can
use gradient descent or linear program



Outlier frequently happens in the real world

Gradient descent to solve linear regression with mean absolute error (MAE) loss function

Except for the Ordinary Least Square (OLS) loss function, we could also construct the mean absolute error (MAE) as the loss function, its formula can be expressed as:

$$L = \frac{1}{n}\sum_{i=1}^{n}|x_i\theta - y_i|$$

Linear regression using MAE loss function is hard to get get the analytical solution. But it is extremely easy using gradient descent alogrithm, much easier than previous shown OLS loss function.

Similarly, we start with the loss function for one single sample:

$$J(\theta; x_i) = |x_i\theta - y_i| = |\sum_{k=1}^{n} x_{ik}\theta_k - y_i|$$

Then the partial derivative for the loss function is:

???

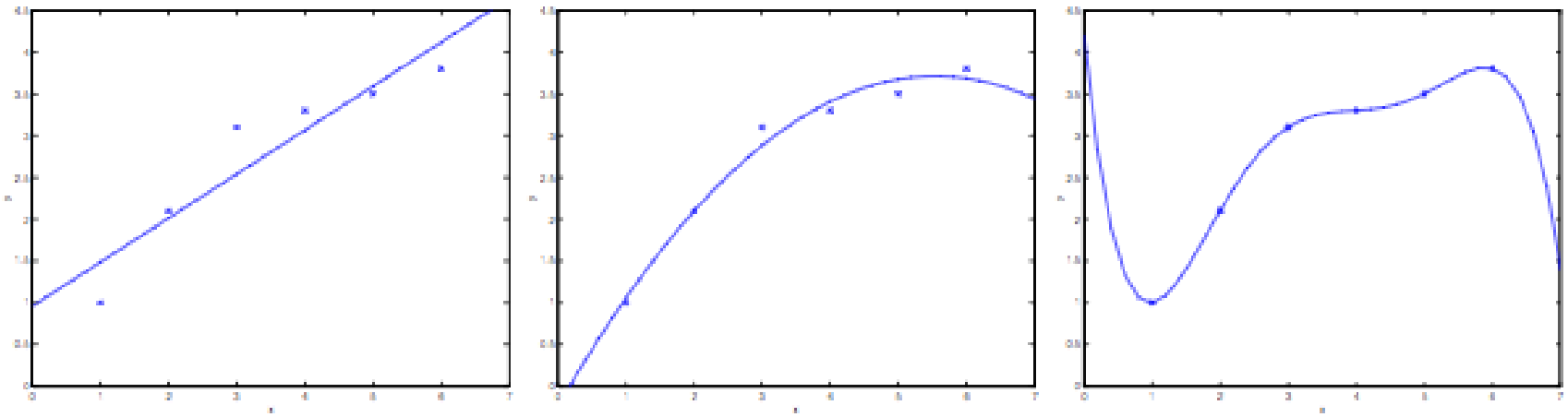
Regression with non-linear features

- If we add extra features that are non-linear
  - For example x^2

Cloth	Corn	Grass	Water	Beer	Rainfall
4	6	3	10	0	**76950**
5	1	0	0	7	**30234**
6	0	3	5	7	**123456**
5	0	3	12	0	**89301**
4	3	0	6	7	**?**

- $h_{\boldsymbol{\theta}}(\mathbf{x_1}) = \theta_0 + \theta_1 x_{1,1} + \theta_2 x_{1,2} + \theta_3 x_{1,3} + \theta_4 x_{1,4} + \theta_5 x_{1,5} + \theta_6 x_{1,1}^2 + \ldots$
- These can be considered as additional features
- We can now have a line that is non-linear

Overfitting Underfitting



Adding more non-linear features makes the line more curvy
(Adding more features also means more model parameters)

The curve can go directly to the outliers with enough parameters.
We call this effect overfitting
For the opposite case, having not enough parameters to model the data is called underfitting

Predicting floods

Cloth	Corn	Grass	Water	Beer	Flood?
4	6	3	10	0	**yes**
5	1	0	0	7	**yes**
6	0	3	5	7	**no**
5	0	3	12	0	**yes**
4	3	0	6	7	**?**

So far, we talk about predicting an amount what if we want to do classification

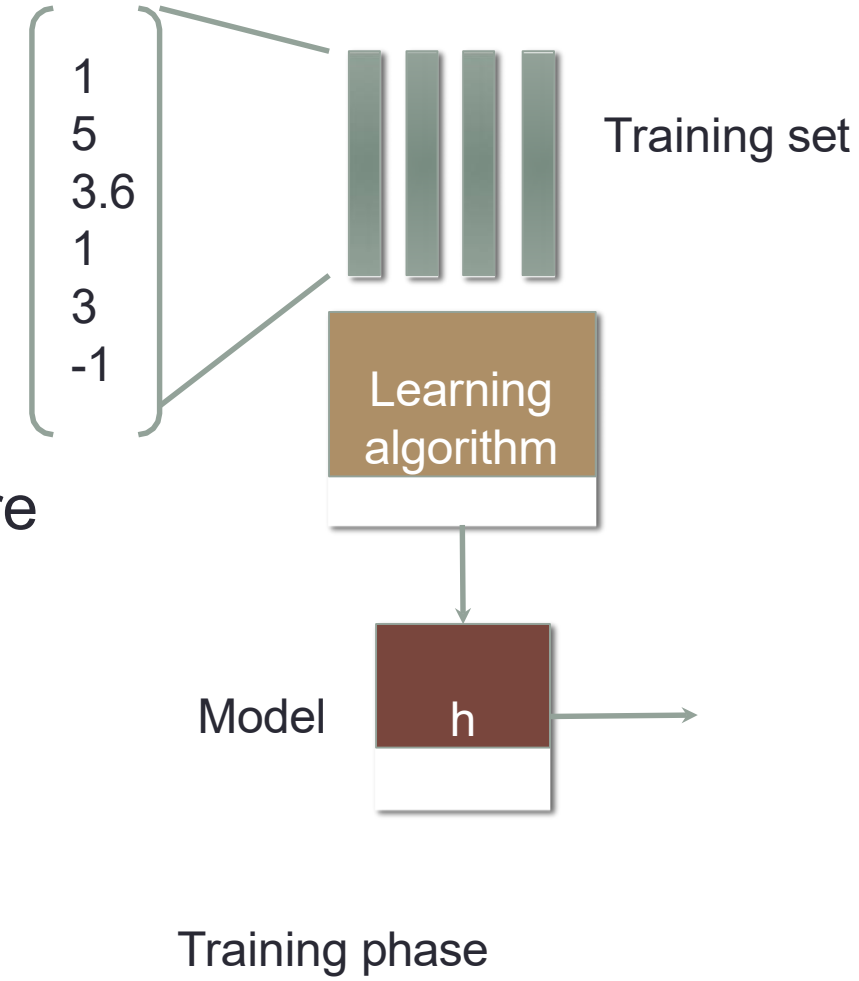
Let’s start with a binary choice. Flood or no flood

Flood or no flood

- What would be the output?

- y = 0 if not flooded
- y = 1 if flooded

- Anything in between is a score for how likely it is to flood



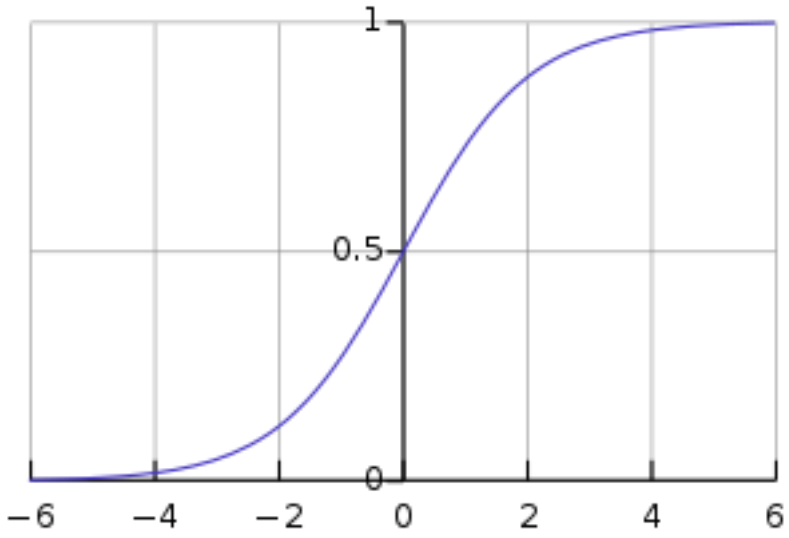
Can we use regression?

- Yes
- $h_{\boldsymbol{\theta}}(\mathbf{x_1}) = \theta_0 + \theta_1 x_{1,1} + \theta_2 x_{1,2} + \theta_3 x_{1,3} + \theta_4 x_{1,4} + \theta_5 x_{1,5}$

- But
- What does it mean when h is higher than 1?
- Can h be negative? What does it mean to have a negative flood value?

Logistic function

- Let's force h to be between 0 and 1 somehow
- Introducing the logistic function (sigmoid function)



$$f(x) = \frac{1}{1+e^{-x}} = \frac{e^{x}}{1+e^{x}}$$

https://en.wikipedia.org/wiki/Logistic_function

Logistic Regression

- Pass $\theta^T \mathbf{x}$ through the logistic function

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Loss function?

- MSE error no longer a good candidate
- The cost function is ‘cross entropy’ error, defining how actual classes match your class predictions.

$$E_i = -(y^{(i)} log(h(\vec{x}^{(i)})) + (1 - y^{(i)}) log(1 - h(\vec{x}^{(i)})))$$

Logistic Regression update rule

$$\theta_j \Leftarrow \theta_j + r\Sigma_{i=1}^{m}(y_i - h_\theta(x_i))x_i^{(j)}$$

Update rule for linear regression

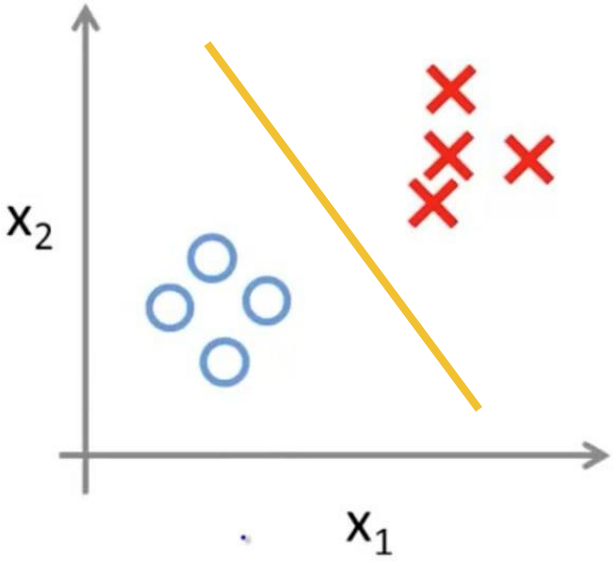
$$\theta_j \Leftarrow \theta_j + r\Sigma_{i=1}^{m}(y_i - \theta^T \mathbf{x}_i)x_i^{(j)}$$

Classification

Classification *is the problem of assigning observations to one or more categories.*

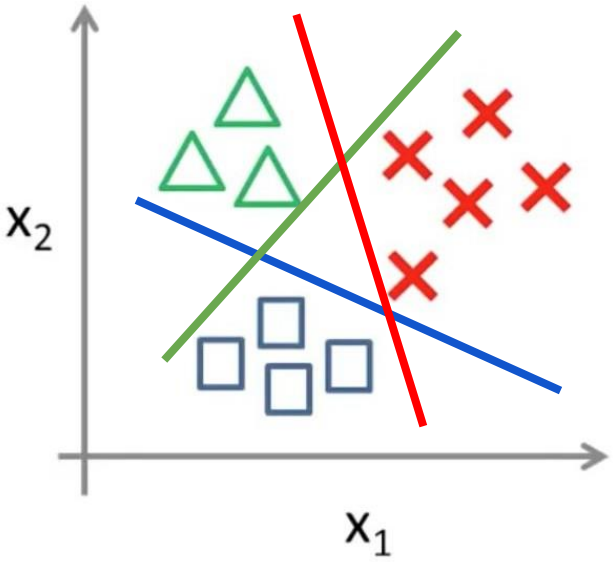
Binary Classification

Involves *only 2* classes



Multiclass Classification

Involves *more than 2* classes



Examples

Binary

- Spam detection
- Churn/no churn customer retention
- Develop Diabetes/don't
- Default/repay loan

Multiclass

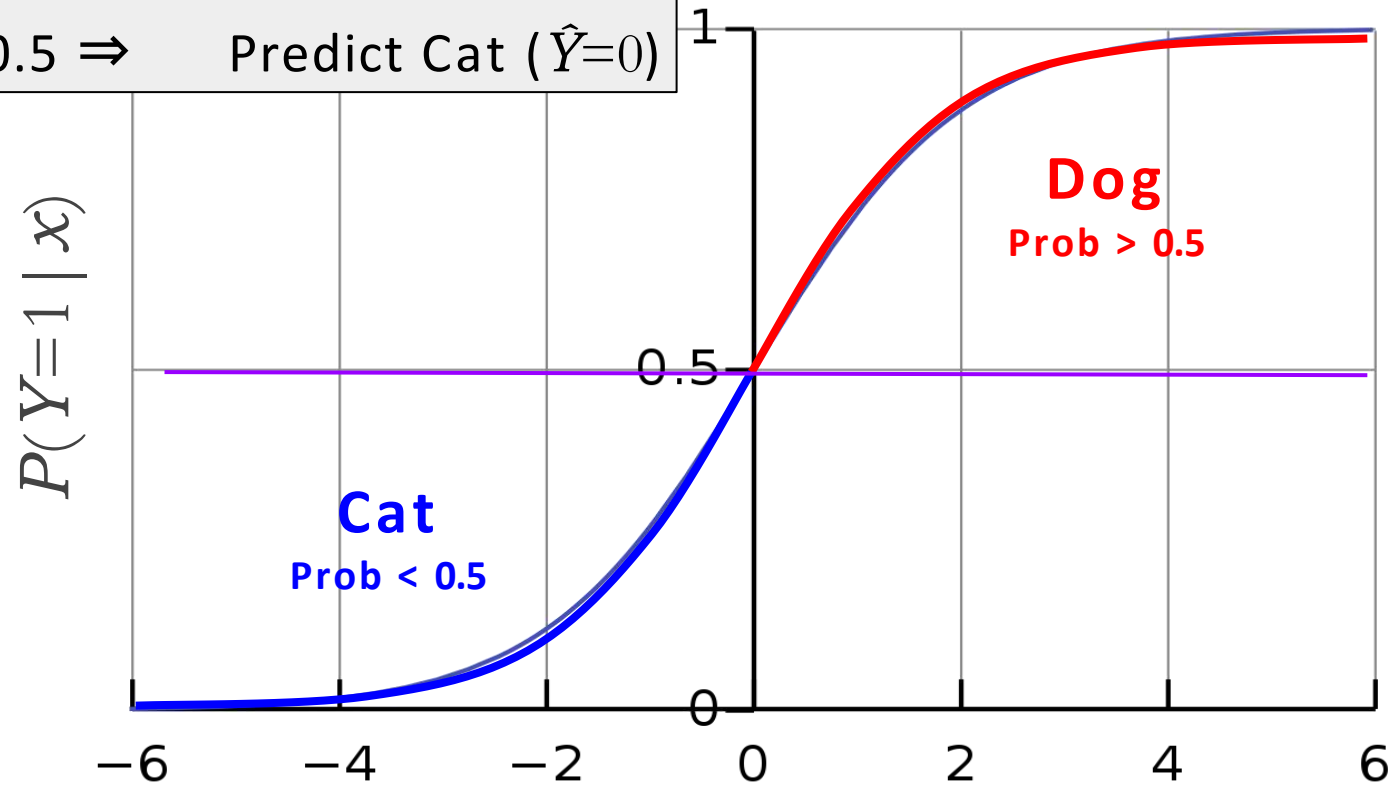
- Image recognition
- Natural language processing

Logistic Regression Threshold

We can select some threshold (**Prob = 0.5**)

- If $P(Y=1 \mid x) > 0.5 \Rightarrow$ Predict Dog ($\hat{Y}=1$)
- If $P(Y=1 \mid x) < 0.5 \Rightarrow$ Predict Cat ($\hat{Y}=0$)

$$P(Y=1|x) = \frac{1}{1+e^{-(w_0+w_1 x)}}$$



Multiclass Classification

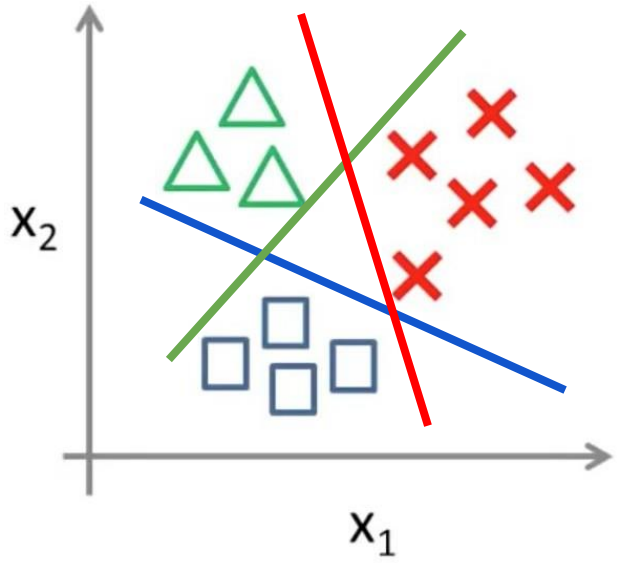
Logistic regression can be applied to solve multiclass problems.

Common Approaches

1. One-vs-Rest (One-vs-All)
2. Softmax Regression (Multinomial Logistic Regression)

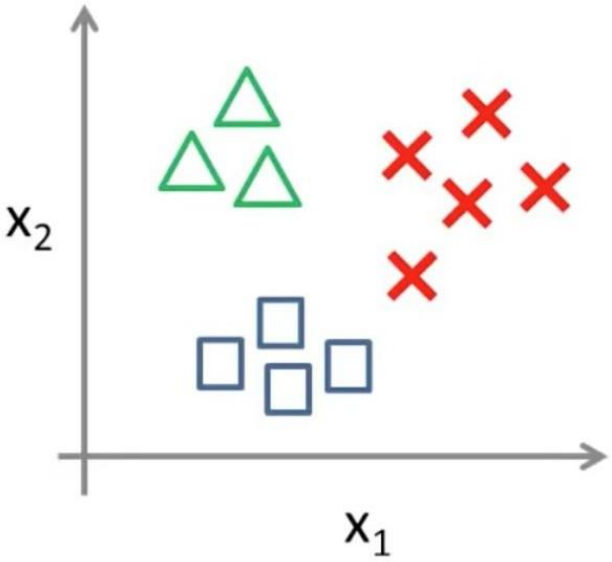
Multiclass Classification

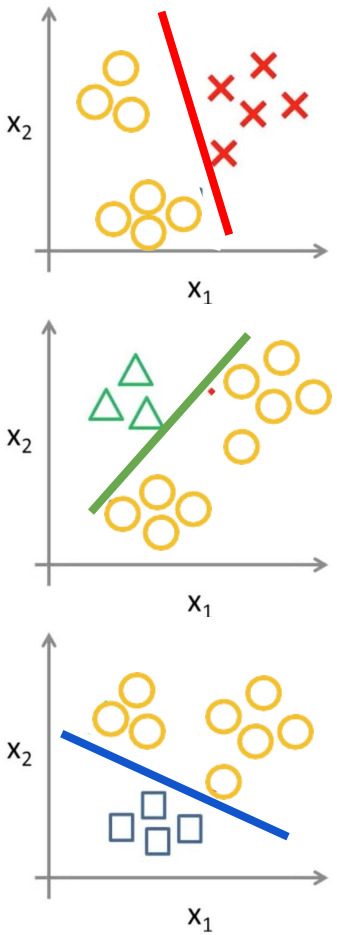
Involves *more than 2* classes



One-vs-Rest (One-vs-All)

For each class, build a logistic regression to find the probability the observation belongs to that class. For each data point, predict the class with the highest probability.





$$P(Y=0 \mid x)$$

$$P(Y=1 \mid x)$$

$$P(Y=2 \mid x)$$

Predict the class with the highest probability

Diagram from https://www.coursera.org/lecture/machine-learning/multiclass-classification-one-vs-all-68Pol

Softmax Regression (Multinomial Logistic Regression)

Softmax Regression Setting

$$x \in \mathbb{R}^d = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

$$W = \begin{bmatrix} (\theta^{(1)})^\top \\ (\theta^{(2)})^\top \\ \vdots \\ (\theta^{(K)})^\top \end{bmatrix} \in \mathbb{R}^{K \times d}$$

$$z = Wx \in \mathbb{R}^K$$

$$\theta^{(j)} \in \mathbb{R}^d = \begin{bmatrix} \theta_1^{(j)} \\ \theta_2^{(j)} \\ \vdots \\ \theta_d^{(j)} \end{bmatrix}$$

x = vector of features

$\theta^{(j)}$ = vector of weights for class *j*

z_j = scalar (score for class *j*)

Softmax takes $z_1, \ldots, z_K$ as input (a vector of size *K*)

There are **d features** in total

$$z_j = \theta^{(j)\top} x = \sum_{i=1}^{d} \theta_i^{(j)} x_i$$

Softmax Regression (Multinomial Logistic Regression)

In softmax regression the probability that a data point belongs to each class is calculated by:

$$\begin{bmatrix} P(Y=1|x;\theta) \\ P(Y=2|x;\theta) \\ \vdots \\ P(Y=K|x;\theta) \end{bmatrix} = \underbrace{\frac{1}{\sum_{j=1}^{K} \exp(\theta^{(j)\top} x)}}_{\text{Normalizes probabilities so they sum to 1.}} \begin{bmatrix} \exp(\theta^{(1)\top} x) \\ \exp(\theta^{(2)\top} x) \\ \vdots \\ \exp(\theta^{(K)\top} x) \end{bmatrix}$$

Predict the class with the highest probability

Separate $\theta^{(j)} \in R^d$ for each class

If $K=2$, softmax regression reduces to the same binary logistic regression formulas we saw earlier. Check out this [overview of softmax regression]() for the proof.

The standard (unit) softmax function $\sigma : \mathbb{R}^K \mapsto (0,1)^K$, where $K \geq 1$, is defined by the formula

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{for } i = 1, \ldots, K \text{ and } \mathbf{z} = (z_1, \ldots, z_K) \in \mathbb{R}^K.$$

Logistic Regression Update Rule for Softmax

Categorical Cross-Entropy

One-hot label

$$y = (y_1, \ldots, y_K), \quad y_k \in \{0, 1\}$$

$$\boxed{L = -\sum_{k=1}^{K} y_k \log P(y = k|x)}$$

Update rule

$$\boxed{\theta^{(k)} \leftarrow \theta^{(k)} - \eta \sum_{i=1}^{m} \left(P(y_i = k|x_i) - y_{ik}\right) x_i}$$

HW1

https://drive.google.com/drive/folders/1KYMAV7RvwGZF4pJ6aNBcb8aRxRz037xd?usp=sharing